



exdqlm: An R Package for Estimation and Analysis of Flexible Dynamic Quantile Linear Models

Antonio De Leon

University of California Santa Cruz

Raquel Barata

University of California Santa Cruz

Raquel Prado

University of California Santa Cruz

Bruno Sansó

University of California Santa Cruz

Abstract

We present the R package **exdqlm** for Bayesian quantile regression, with primary emphasis on dynamic state-space quantile models for time series. The package is built around extended dynamic quantile linear models (exDQLMs), which use the extended asymmetric Laplace (exAL) family, a parametric extension of the asymmetric Laplace (AL) distribution commonly used in quantile regression. The software provides posterior simulation via Markov chain Monte Carlo (MCMC) and fast approximate posterior inference via Laplace–delta variational Bayes (LDVB), supporting posterior uncertainty quantification while also providing a computationally efficient option for longer time series. The same package interface supports static exAL quantile regression with regularized priors, dynamic transfer-function models for nonlinear input effects at a given quantile, post hoc posterior-predictive synthesis across separately fitted quantiles, forecasting, and quantitative and visual diagnostics for model evaluation.

Keywords: dynamic quantile regression, dynamic models, asymmetric Laplace, Laplace–delta variational Bayes, coordinate ascent variational inference, quantile regression.

1. Introduction

Quantile regression has become a widely used alternative to mean-based modeling in static settings. However, dynamic quantile methods and accompanying software remain comparatively limited. In Bayesian parametric quantile regression, the asymmetric Laplace (AL) likelihood has played a central role because it is linked to the check-loss function used in classical quantile regression (Yu and Moyeed 2001; Koenker 2005) and allows convenient mixture

representations for posterior computation. Nevertheless, the AL distribution can be restrictive as an error model. More flexible error distributions have been studied extensively in the Bayesian nonparametric literature (Kottas and Krnjajić 2009; Reich, Bondell, and Wang 2009; Taddy and Kottas 2010); however, the resulting methods can be computationally demanding for dynamic models and longer time series.

Yan, Zheng, and Kottas (2025) introduce the extended asymmetric Laplace (exAL) distribution, a parametric extension of the AL distribution that preserves many of its computational advantages while relaxing some restrictive assumptions. Their work also develops Bayesian exAL regression for static quantile models. Building on the exAL family, Barata, Prado, and Sansó (2022) developed the extended dynamic quantile linear model (exDQLM) framework for modeling time-varying quantiles, together with Markov chain Monte Carlo (MCMC) and importance-sampling variational Bayes (ISVB) inference and a transfer-function extension for lagged covariate effects. The Laplace–delta variational Bayes (LDVB) implementation considered here follows the general nonconjugate variational inference strategy of Wang and Blei (2013), specialized to the exAL scale–skewness block; in version 1.1.1, the default LDVB scale–skewness factor is a structured $q(\gamma)q(\sigma \mid \gamma)$ approximation that keeps the conditional GIG scale structure explicit. Thus, the exAL distribution, the exDQLM state-space model formulation, the MCMC and ISVB algorithms in Barata *et al.* (2022), and the general nonconjugate variational strategy constitute the principal methodological foundations behind the present work.

The contribution of the present article is software-centered. The R package **exdqlm** turns these methodological components into an integrated workflow for specifying, fitting, inspecting, diagnosing, forecasting, and reproducing Bayesian dynamic and static quantile analyses. The package makes five main software contributions:

- a composable model-construction interface for trend, seasonal, regression, and transfer-function components in Bayesian dynamic quantile state-space models;
- coordinated MCMC and variational inference interfaces, including scale-collapsed MCMC, LDVB, and ISVB, for dynamic and static fits;
- an S3 object workflow for fitted models, post-processing and forecasting objects with `print()`, `summary()`, `plot()`, `predict()`, and `diagnostics()` methods where appropriate;
- package-level exAL distribution utilities, predictive scoring and calibration diagnostics, and compiled implementations of computationally intensive model-building, posterior sampling and posterior predictive operations, with R fallbacks where applicable; and
- support for static AL/exAL regression with shrinkage-prior options, together with scripts and documentation that reproduce the analyses reported in this article.

The R (R Core Team 2026) software ecosystem for quantile regression is broad, but the capabilities most relevant to this paper are distributed across software with different aims. Table 1 compares representative packages along four traits that are key to the scope of **exdqlm**: the primary modeling objective, the treatment of the temporal or latent-state structure, the inference strategy, and handling of multiple quantile levels. The distinction between time included as an observed predictor and coefficients propagated as latent states is particularly

important. The computation column reports how fitting, posterior approximation, or state summaries are obtained, rather than treating these as additional model structures.

Package	Main scope	Temporal or state-space handling	Inference or computation	Multiple-quantile handling
quantreg	General frequentist QR	Time-series regression formula: <code>dynrq()</code> handles lags, differences, trends, and seasons; no latent states	Optimization-based fitting; frequentist and bootstrap summaries	Vector-valued <code>tau</code> ; post hoc rearrangement
bayesQR	Bayesian AL QR	Time or lags only as user-supplied covariates; no serial or state-space model	MCMC under the AL likelihood; optional adaptive lasso	Vector-valued quantiles; output by fitted quantile
pqrBayes	Bayesian penalized QR	Varying-coefficient regression: coefficients are B-spline functions of an observed modifier, possibly time; no serial or state-space model	MCMC with shrinkage priors	One target quantile per fit
brms	General Bayesian regression with an AL family	Regression autocorrelation terms can be added; no quantile state-space workflow	Stan/NUTS posterior sampling	Fixed through the <code>asym_laplace()</code> quantile parameter
qgam	Smooth additive QR	Time may enter as a smooth covariate; no serial or state-space model	Calibrated Gibbs posterior with GAM fitting	Single quantile via <code>qgam()</code> ; multiple via <code>mqgam()</code>
qrjoint	Bayesian joint linear QR	Time only as a predictor; no serial or state-space model	Adaptive MCMC for joint quantile planes	Joint noncrossing quantile planes
dlm	Gaussian state-space modeling	Gaussian latent states with filtering, smoothing, and forecasting; not QR	Kalman state recursions; MLE or Bayesian parameter analysis	Not a quantile-regression package
exdqglm	Bayesian exAL QR	Quantile latent states with filtering/smoothing summaries, forecasting, and transfer functions	MCMC posterior simulation; VB approximate posterior inference	Post hoc synthesis from separately fitted quantiles

Table 1: R packages relevant to quantile and state-space modeling. Entries summarize documented primary interfaces along the dimensions used to position **exdqglm**; they are selective rather than exhaustive. In the temporal column, time-as-predictor/smooth/modifier uses, regression autocorrelation, Gaussian latent states, and quantile latent states are distinct categories. QR denotes quantile regression, AL asymmetric Laplace, exAL its extended version, NUTS the No-U-Turn sampler, GAM generalized additive model, MLE maximum likelihood estimation, and VB variational Bayes.

The frequentist package **quantreg** (Koenker 2025) remains the broad general-purpose baseline, covering linear, nonlinear, nonparametric, censored, and dynamic quantile regression,

together with bootstrap-based and related uncertainty summaries. Its `dynrq()` function supports dynamic linear quantile regression through time-series regression formulas involving lags, differences, trends, seasonal factors, and harmonic terms. However, the dynamic structure enters through the design matrix rather than through latent states. Once these regressors have been constructed, estimation proceeds through the standard **quantreg** fitting routines. Thus, `dynrq()` is a regression-style time-series interface rather than one for a latent state-space model. Most other R packages relevant here implement static quantile regression or target a different multi-quantile objective. Among the static Bayesian options, **bayesQR** (Benoit, Al-Hamzawi, Yu, and Van den Poel 2023) provides Bayesian quantile regression under the AL working likelihood with an adaptive-lasso option, whereas **pqrBayes** (Fan, Wu, Ren, Li, and Zhou 2026) emphasizes penalized Bayesian quantile regression with spike-and-slab and horseshoe-family shrinkage priors. The varying-coefficient option, specified by `model = "VC"` in **pqrBayes**, models each regression coefficient as a smooth function of a user-supplied observed modifier. It represents these functions using a B-spline basis and estimates the corresponding basis coefficients using MCMC under shrinkage priors. Thus, **pqrBayes** is appropriate for static varying-coefficient quantile regression, including applications where the chosen modifier is calendar time or another time index. Even in such applications, however, the coefficients are modeled as smooth functions of time rather than propagated through a latent state equation. The broader Bayesian modeling package **brms** (Bürkner 2026) is also relevant because it supports AL-based quantile regression within a flexible multilevel, nonlinear, and additive regression framework, and can incorporate regression-level autocorrelation terms. However, it is not organized as a dedicated quantile state-space workflow, and its documentation does not present filtered or smoothed state summaries of the type targeted by **exdq1m**. Packages such as **qgam** (Fasiolo and Griffiths 2025), **qrjoint** (Tokdar and Cunningham 2025), **quantregGrowth** (Muggeo 2025), and **qrcm** (Frumento 2025) address additive, joint, non-crossing, or coefficient-modeling formulations. Other packages focus on specialized static settings, including penalized frequentist fitting across quantiles in **rqPen** (Sherwood, Li, and Maidman 2026), convolution-smoothed linear quantile regression with efficient gradient-based fitting in **conquer** (He, Pan, Tan, and Zhou 2023), and robust or censored-response modeling in **lqr** (Galarza, Benites, Bourguignon, and Lachos 2024).

Quantile-regression software implementations are also available in Python, MATLAB and Julia, but they are general static quantile regression tools and do not provide the Bayesian latent-state quantile workflow considered here. **exdq1m** is not intended to replace broad general-purpose quantile-regression toolkits, additive quantile-regression packages, or software designed specifically for joint multi-quantile estimation. Its role is more specific: it fills a gap between general quantile-regression software and general Gaussian state-space software by providing a dedicated Bayesian workflow for dynamic quantile state-space models. General state-space packages in R, such as **d1m** (Petris and Gilks 2018) and **dynr** (Ou, Hunter, Chow, Ji, Chen, Hung, Lee, Li, and Park 2021), remain useful for dynamic modeling, but they do not provide the quantile likelihoods, filtered and smoothed quantile-state summaries, diagnostics, forecasting, or synthesis tools targeted here.

The **exdq1m** package is distributed under the MIT License and is available from the Comprehensive R Archive Network at <https://CRAN.R-project.org/package=exdq1m>. The package source submitted with this article is version 1.1.1, a reproducibility patch built on the public 1.1.x package line.

The remainder of the paper is organized as follows. Section 2 summarizes the modeling frame-

work, including algorithms used for posterior inference and transfer-function state augmentation for nonlinear input effects. It also discusses static quantile regression as a special case of the framework. Section 2.2 details the model diagnostics and forecasting method implemented in the package. Section 3 describes the package design and implementation, including the object families and standard methods used by the examples. Section 4 illustrates the package through four examples. Section 5 summarizes the package capabilities, and Appendices A–F collect selected posterior targets, package-specific computational blocks, diagnostic formulas, and predictive-synthesis details used by the implementation.

2. Extended dynamic quantile linear models

This section provides an overview of extended dynamic quantile linear models (exDQLMs) along with related posterior inference and tools for model diagnostics. Additional implementation details and posterior derivations are provided in Appendices A–F.

2.1. General modeling framework

Model formulation

Let y_t denote a scalar response observed at times $t = 1, \dots, T$. For each t , an extended dynamic quantile linear model (exDQLM) for the p_0 quantile is defined by

$$\begin{aligned} \text{Observation equation:} \quad y_t &= \mathbf{F}_t^\top \boldsymbol{\theta}_t + \epsilon_t, & \epsilon_t &\sim \text{exAL}_{p_0}(0, \sigma, \gamma) \\ \text{System equation:} \quad \boldsymbol{\theta}_t &= \mathbf{G}_t \boldsymbol{\theta}_{t-1} + \boldsymbol{\omega}_t, & \boldsymbol{\omega}_t &\sim \mathcal{N}(\mathbf{0}, \mathbf{W}_t). \end{aligned} \quad (1)$$

Here $\mathbf{F}_t$ is the $q \times 1$ regression vector of the covariates, $\boldsymbol{\theta}_t$ is the corresponding $q \times 1$ latent state vector and $\mathbf{G}_t$ is the $q \times q$ -dimensional evolution matrix. The state innovation $\boldsymbol{\omega}_t$ follows a q -variate normal distribution with state covariance matrix $\mathbf{W}_t$. The observational errors ϵ_t follow an extended asymmetric Laplace (exAL) distribution for fixed quantile p_0 , whose density is denoted as $\text{exAL}_{p_0}(\cdot)$, and it is such that $\int_{-\infty}^0 \text{exAL}_{p_0}(\epsilon_t | 0, \sigma, \gamma) d\epsilon_t = p_0$. Thus, the observation equation in (1) implies that $\mathbf{F}_t^\top \boldsymbol{\theta}_t$ corresponds to the p_0 -quantile of y_t . Table 2 shows how the state-space quantities and prior hyperparameters are represented in the `exdqim` function inputs.

The exAL distribution, introduced by Yan *et al.* (2025), extends the location-scale mixture representation of the AL (Kozumi and Kobayashi 2011). For fixed p_0 , the skewness parameter γ is restricted to an interval (L, U) , where L and U are the negative and positive roots of $g(\gamma) = 1 - p_0$ and $g(\gamma) = p_0$, respectively, with $g(\gamma) = 2\Phi(-|\gamma|) \exp(\gamma^2/2)$. This yields the following mixture representation of $\text{exAL}_{p_0}(0, \sigma, \gamma)$:

$$\begin{aligned} \text{exAL}_{p_0}(y_t | 0, \sigma, \gamma) &= \int \int_{\mathbb{R}^+ \times \mathbb{R}^+} \mathcal{N}(y_t | C(p, \gamma)\sigma|\gamma|s_t + A(p)v_t, \sigma B(p)v_t) \\ &\quad \times \text{Exp}(v_t | \sigma) \mathcal{N}^+(s_t | 0, 1) dv_t ds_t. \end{aligned} \quad (3)$$

Here, $p = p(p_0, \gamma) = I(\gamma < 0) + \{[p_0 - I(\gamma < 0)]/g(\gamma)\}$ where $g(\gamma) = 2\Phi(-|\gamma|) \exp(\gamma^2/2)$, $\Phi(\cdot)$ denotes the standard normal CDF, and $I(\cdot)$ is the indicator function. $A(p)$, $B(p)$, $C(p, \gamma)$ are the functions of p and γ : $A(p) = \frac{1-2p}{p(1-p)}$, $B(p) = \frac{2}{p(1-p)}$, $C(p, \gamma) = [I(\gamma > 0) - p]^{-1}$. Lastly,

$\text{Exp}(v \mid \sigma)$ denotes the exponential distribution with mean σ , and $\mathcal{N}^+(s_t \mid 0, 1)$ denotes a normal distribution truncated to the positive reals with mean 0 and variance 1. Introducing the latent variables v_t and s_t gives the hierarchical representation:

$$y_t \mid \boldsymbol{\theta}_t, \sigma, \gamma, v_t, s_t \sim \mathcal{N}(y_t \mid \mathbf{F}_t^\top \boldsymbol{\theta}_t + C(p, \gamma) \sigma |\gamma| s_t + A(p) v_t, \sigma B(p) v_t) \quad (4)$$

$$v_t, s_t \mid \sigma \sim \text{Exp}(v_t \mid \sigma) \mathcal{N}^+(s_t \mid 0, 1) \quad (5)$$

$$\boldsymbol{\theta}_t \mid \boldsymbol{\theta}_{t-1}, \mathbf{W}_t \sim \mathcal{N}(\mathbf{G}_t \boldsymbol{\theta}_{t-1}, \mathbf{W}_t). \quad (6)$$

The posterior algorithms in **exdq_{lm}** use this hierarchical form.

Prior specification

Conditional on p_0 , the model assigns priors to the initial state $\boldsymbol{\theta}_0$, scale parameter σ and skewness parameter γ . For $\boldsymbol{\theta}_0$, we assume a q -variate conjugate normal prior $\boldsymbol{\theta}_0 \sim \mathcal{N}(\mathbf{m}_0, \mathbf{C}_0)$ and the scale parameter follows a conjugate inverse-gamma prior, $\sigma \sim \text{IG}(a_\sigma, b_\sigma)$. The parameter γ has bounded support over the interval (L, U) where L and U are the negative and positive roots associated with p_0 in the exAL representation (Yan *et al.* 2025). The package uses a proper Student- t prior on the skewness parameter, truncated to the admissible interval (L, U) , i.e., $\gamma \sim t_{(L, U)}(m_\gamma, s_\gamma)$ with ν_γ degrees of freedom, which improves numerical stability while respecting the support restrictions from the exAL family. Table 2 gives the corresponding package inputs and default values.

R input	model				PriorSigma		PriorGamma		
Mathematical symbol	$\mathbf{F}_{1:T}$	$\mathbf{G}_{1:T}$	$\mathbf{m}_0$	$\mathbf{C}_0$	a_σ	b_σ	m_γ	s_γ	ν_γ
R component	FF	GG	m0	C0	a_sig	b_sig	m_gam	s_gam	df_gam
Default if omitted	–	–	$\mathbf{0}_q$	$10^3 \mathbf{I}_q$	2.1	1.1	0	1	1

Table 2: Translation from exDQLM notation to **exdq_{lm}** inputs. The **model** object stores state-space components and the initial-state prior, while **PriorSigma** and **PriorGamma** store hyperparameters for σ and γ . A dash indicates no generic default because FF and GG are determined by the selected model components. Here $\mathbf{0}_q$ and $\mathbf{I}_q$ denote a q -vector of zeros and the q -dimensional identity matrix.

Posterior estimation

The dynamic exDQLM posterior target and the original MCMC and ISVB inference strategy were developed by Barata *et al.* (2022). The package makes **exdq_{lm}ISVB()** available for variational inference following the ISVB approach of Barata *et al.* (2022). The principal fitting routines in the package are **exdq_{lm}MCMC()**, which performs posterior inference via MCMC, and **exdq_{lm}LDVB()**, which provides a faster variational approximation based on the nonconjugate variational strategy of Wang and Blei (2013).

The MCMC routine uses Gibbs sampling steps to update the latent-variable sequences $\{s_t\}_{t=1}^T$ and $\{v_t\}_{t=1}^T$. The dynamic state sequence $\boldsymbol{\theta}_{1:T}$ is sampled using a forward-filtering backward-sampling algorithm (Carter and Kohn 1994; Frühwirth-Schnatter 1994). The scale and skewness parameters are updated separately from the latent-state blocks. By default, the package integrates σ out of the conditional target for γ , updates γ on its bounded logit scale using a univariate slice sampler (Neal 2003), and then redraws σ from its exact GIG conditional distribution. Joint random-walk Metropolis–Hastings (MH) alternatives are also available. The

chain can optionally be initialized using the variational approximation, but this initialization is separate from the subsequent MCMC update kernel.

For longer time series or repeated model comparisons, MCMC can be computationally demanding. The `exdq1mLDVB()` routine uses a mean-field factorization and blockwise coordinate-ascent variational inference. The blocks associated with the latent variables and dynamic states admit closed-form coordinate updates, whereas the joint scale-skewness block is non-conjugate. By default, `exdq1m` uses a structured $q(\gamma)q(\sigma | \gamma)$ factor for this block: the marginal factor for γ is represented on its bounded logit scale, and the conditional factor for σ uses the GIG kernel available at each γ . The previous two-dimensional Laplace-delta approximation on transformed (σ, γ) coordinates remains available as a legacy option. The dynamic states are updated at each iteration using forward-filtering backward-smoothing, and convergence is monitored through changes in the state, scale, skewness, and evidence lower bound (ELBO).

Relative to the ISVB implementation, LDVB replaces the stochastic importance-sampling approximation for the non-conjugate scale-skewness block with a deterministic variational approximation conditional on the data and tuning inputs. This leads to reproducible convergence monitoring and provides approximate uncertainty summaries for both σ and γ . After convergence, samples from the fitted variational factors are generated for posterior summarization. These are approximate variational draws rather than draws from the exact posterior, so MCMC remains the reference calculation when posterior tail behavior or interval accuracy is central.

Table 3 shows selected core inference controls for the fitting functions `exdq1mMCMC()` and `exdq1mLDVB()`. The MCMC routine also accepts `Sig.mh` as an optional proposal covariance when `mh.proposal` selects a joint random-walk MH kernel; it is not used under the default scale-collapsed slice update. Both fitting routines also use package-level options to control selected compiled and R backend paths. These runtime options, including ELBO monitoring where applicable, are described in Section 3 and are distinct from the function-specific inference controls summarized in Table 3.

Control	exdq1mMCMC()			exdq1mLDVB()	
Mathematical symbol	$\mathcal{K}_{\sigma,\gamma}$	N_{BURN}	N_{MCMC}	ϵ_{tol}	N_{SAMP}
R argument	mh.proposal	n.burn	n.mcmc	tol	n.samp
Default if omitted	"slice"	2000	1500	0.1	200

Table 3: Core inference controls for `exdq1mMCMC()` and `exdq1mLDVB()`. $\mathcal{K}_{\sigma, \gamma}$ denotes the update kernel for the (σ, γ) block, N_{BURN} is the burn-in length, N_{MCMC} is the number of retained MCMC draws, ϵ_{tol} is the LDVB convergence tolerance and N_{SAMP} is the number of requested variational draws. Advanced control-list arguments and backend options are documented separately.

Evolution covariance and discount factors

Both `exdq1mMCMC()` and `exdq1mLDVB()` use discount factors to specify the evolution covariance matrices $\mathbf{W}_t$ in Equation (2) (see West and Harrison 1997; Prado, Ferreira, and West 2021, and references therein). Under a common discount factor $\delta \in (0, 1]$, $\mathbf{W}_t = \frac{1-\delta}{\delta} \mathbf{G}_t \mathbf{C}_{t-1} \mathbf{G}_t^\top$ where $\mathbf{C}_{t-1}$ denotes the posterior variance of the state vector $\boldsymbol{\theta}_{t-1}$ at time

$t - 1$. The discount factor δ can be specified by the user via the function parameter **df**. The selection of discount factors can be done by optimizing a given selection criterion such as the continuous ranked probability score (CRPS) or the posterior predictive loss criterion (PPLC) defined in Section 2.2, with Kullback–Leibler (KL) divergence as a complementary calibration diagnostic. Discount factor selection is illustrated in Section 4.2.

Component-specific discounting is also available. When the state vector is partitioned into h components with dimensions $q_1, \dots, q_h$, each block of the evolution covariance matrix can be assigned its own discount factor. The arguments **df** and **dim.df** specify the component discount factors $(\delta_1, \dots, \delta_h)$ and their corresponding dimensions $(q_1, \dots, q_h)$. Section 4 provides examples. West and Harrison (1997) and Prado *et al.* (2021) give a detailed treatment of component discounting.

Transfer function models

Transfer functions provide a state-space representation for current and lagged input effects on a response quantile. In **exdqmlm**, this is handled by augmenting the dynamic state-space representation conditional on a fixed transfer-function rate λ , and the resulting workflow is available through both LDVB and MCMC fitting routines. For a transfer input vector $\mathbf{x}_t \in \mathbb{R}^k$, a transfer-function exDQLM with exponential decay is defined by

$$y_t | \boldsymbol{\theta}_t, \zeta_t, \gamma, \sigma \sim \text{exAL}_{p_0}(\mathbf{F}_t^\top \boldsymbol{\theta}_t + \zeta_t, \sigma, \gamma) \quad (7)$$

$$\boldsymbol{\theta}_t | \boldsymbol{\theta}_{t-1}, \mathbf{W}_t^\theta \sim \mathcal{N}(\mathbf{G}_t \boldsymbol{\theta}_{t-1}, \mathbf{W}_t^\theta) \quad (8)$$

$$\zeta_t | \zeta_{t-1}, \boldsymbol{\psi}_{t-1}, \omega_t \sim \mathcal{N}(\lambda \zeta_{t-1} + \mathbf{x}_t^\top \boldsymbol{\psi}_{t-1}, \omega_t) \quad (9)$$

$$\boldsymbol{\psi}_t | \boldsymbol{\psi}_{t-1}, \mathbf{W}_t^\psi \sim \mathcal{N}(\boldsymbol{\psi}_{t-1}, \mathbf{W}_t^\psi). \quad (10)$$

Here $\zeta_t \in \mathbb{R}$ denotes the accumulated transfer effect on the quantile at time t and $\boldsymbol{\psi}_t \in \mathbb{R}^k$ is the vector of instantaneous transfer coefficients associated with the components of $\mathbf{x}_t$. The mean contribution of the transfer block is therefore $\lambda \zeta_{t-1} + \mathbf{x}_t^\top \boldsymbol{\psi}_{t-1}$: the inner product $\mathbf{x}_t^\top \boldsymbol{\psi}_{t-1}$ captures the contemporaneous effect of the current inputs, and $\lambda \in (0, 1)$ controls the geometric decay of previous accumulated effects. More explicitly, the contribution of $\mathbf{x}_t$ to the quantile at time $t + h$ is $\lambda^h a_t$, where $a_t = \mathbf{x}_t^\top \boldsymbol{\psi}_{t-1}$. For tolerance $\epsilon > 0$, the package summarizes the continuous decay horizon

$$k_t = \begin{cases} \{\log(\epsilon) - \log(|a_t|)\} / \log(\lambda), & |a_t| > \epsilon, \\ 0, & |a_t| \leq \epsilon, \end{cases}$$

whose positive values solve $\lambda^{k_t} |a_t| = \epsilon$. The reported **median.kt** is the median of these horizons over time for $\epsilon = 1\text{e-}3$.

Conditional on a fixed value $\tilde{\lambda}$, the dynamic fitting routines incorporate the transfer-function structure by replacing $\mathbf{F}_t$, $\boldsymbol{\theta}_t$, $\mathbf{G}_t$, and $\mathbf{W}_t^\theta$ in Equations (1)-(2) with

$$\begin{aligned} \tilde{\mathbf{F}}_t^\top &= (\mathbf{F}_t^\top, 1, \mathbf{0}_k^\top), & \tilde{\boldsymbol{\theta}}_t^\top &= (\boldsymbol{\theta}_t^\top, \zeta_t, \boldsymbol{\psi}_t^\top), \\ \tilde{\mathbf{G}}_t &= \text{blockdiag}\{\mathbf{G}_t, \mathbf{G}_t^{tf}\}, & \mathbf{G}_t^{tf} &= \begin{pmatrix} \tilde{\lambda} & \mathbf{x}_t^\top \\ \mathbf{0}_k & \mathbf{I}_k \end{pmatrix}, \end{aligned}$$

and $\tilde{\mathbf{W}}_t = \text{blockdiag}\{\mathbf{W}_t^\theta, \omega_t, \mathbf{W}_t^\psi\}$. This augmentation is implemented in both functions **exdqmlmTransferLDVB()** and **exdqmlmTransferMCMC()**. Discount factors are used to specify the

transfer block variances ω_t and $\mathbf{W}_t^{\psi}$. In the package, `tf.df` can be supplied as a single shared discount factor, as a pair $(\delta_\zeta, \delta_\psi)$ with a shared value for the whole ψ_t block, or componentwise across $(\zeta_t, \psi_{1,t}, \dots, \psi_{k,t})$. A conjugate normal prior on $(\zeta_0, \psi_0^\top)^\top \sim \mathcal{N}(\mathbf{m}_0^{tf}, \mathbf{C}_0^{tf})$, with $\mathbf{m}_0^{tf} \in \mathbb{R}^{k+1}$ and $\mathbf{C}_0^{tf} \in \mathbb{R}^{(k+1) \times (k+1)}$, completes the transfer-function model extension. Table 4 shows the transfer-function arguments.

Quantity	Mathematical symbol	R argument	Input/default
Transfer inputs	$\mathbf{x}_t$	<code>X</code>	Required numeric vector or $T \times k$ matrix.
Transfer rate	$\tilde{\lambda}$	<code>lam</code>	Required scalar in $(0, 1)$.
Discount factors	$(\delta_\zeta, \delta_\psi)$	<code>tf.df</code>	Required vector: length 1 (shared); length 2 (δ_ζ and shared δ_ψ); or length $k+1$ (componentwise).
Prior mean	$\mathbf{m}_0^{tf}$	<code>tf.m0</code>	Optional vector of length $k+1$; default $\mathbf{0}_{k+1}$.
Prior covariance	$\mathbf{C}_0^{tf}$	<code>tf.C0</code>	Optional $(k+1) \times (k+1)$ covariance matrix; default $\mathbf{I}_{k+1}$.

Table 4: Transfer-function inputs for `exdqlmTransferLDVB()` and `exdqlmTransferMCMC()`. Here k denotes the number of transfer covariates.

Special cases

Several commonly used models are special cases of the `exDQLM`. First, setting $\gamma = 0$ reduces the `exAL` observation distribution to the `AL` distribution, i.e., $\epsilon_t \sim \text{exAL}_{p_0}(0, \sigma, \gamma = 0) = \text{AL}_{p_0}(0, \sigma)$, leading to the dynamic quantile linear model (DQLM) of Gonçalves, Migon, and Bastos (2017). The DQLM can be implemented with our package using the parameter `dqlm.ind = TRUE`.

Second, setting $\mathbf{G}_t = \mathbf{I}$ and $\mathbf{W}_t = \mathbf{0}$ reduces the model to the static `exAL` regression of Yan *et al.* (2025). This model can be implemented using functions `exalStaticMCMC()` or `exalStaticLDVB()`.

Finally, combining these two restrictions gives static Bayesian linear quantile regression under the `AL` working likelihood Yu and Moyeed (2001), as implemented in packages such as `bayesQR`. Within the `exdqlm` package, this model can be fitted using `exalStaticMCMC()` or `exalStaticLDVB()` with `al.ind = TRUE`, a static convenience alias for `dqlm.ind = TRUE`, which fixes $\gamma = 0$. Examples of these special models are given in Section 4, with the sparse static regression workflow illustrated in Section 4.4.

2.2. Model diagnostics and forecasting

The package includes quantitative and graphical diagnostics for checking fitted quantile calibration, predictive performance, and forecasts. Calibration is assessed using plug-in one-step-ahead probability integral transforms and standardized forecast errors. Predictive performance is summarized using CRPS and a target-quantile posterior-predictive loss criterion, while future quantiles and optional posterior-predictive draws are obtained through the standard `predict()` interface.

Calibration diagnostics

The one-step-ahead predictive probability integral transform (PIT) introduced by Rosenblatt

(1952) is a useful diagnostic for models in which the marginal forecast distributions are not available in closed-form (Huerta, Jiang, and Tanner 2003; Prado, Molina, and Huerta 2006). Let Y_t be the random response at time t with observed value y_t and let $y_{1:(t-1)}$ denote the observations available up to time $t - 1$. The one-step-ahead PIT is

$$u_t^* = \Pr(Y_t \leq y_t \mid y_{1:t-1}), \quad (11)$$

Under the true conditional distribution of Y_t given the past at every time, the PIT values $\{u_t^*\}$ are independent $\text{Uniform}(0, 1)$ random variables (Rosenblatt 1952).

The package reports a maximum a posteriori (MAP) plug-in version based on the conditional Gaussian filtering representation. If $\hat{f}_t$ and $\hat{q}_t$ denote the MAP plug-in one-step-ahead forecast location and variance, the stored standardized forecast error and the corresponding PIT diagnostic are

$$\hat{e}_t = \frac{y_t - \hat{f}_t}{\sqrt{\hat{q}_t}}, \quad \text{and} \quad \hat{u}_t = \Phi(\hat{e}_t), \quad (12)$$

where Φ denotes the standard normal CDF. These are diagnostic summaries of the fitted one-step-ahead sequence rather than posterior uncertainty bands for residuals. Serial dependence in $\{\hat{u}_t\}$ is examined using an autocorrelation function (ACF) plot. The normal transformation $\Phi^{-1}(\hat{u}_t)$, which corresponds to $\hat{e}_t$ under the plug-in Gaussian representation is assessed using a normal quantile–quantile (QQ) plot.

Departure from normality is also summarized using the KL divergence (Kullback and Leibler 1951). If P_e denotes the density estimate of the MAP standardized forecast-error distribution and ϕ denotes the standard normal density, the package reports a forward diagnostic, denoted $\text{KL}(P_e \parallel \phi)$ and, as a sensitivity measure, the reversed diagnostic $\text{KL}(\phi \parallel P_e)$. Smaller values suggest better calibration. These MAP standardized forecast errors are included in the output of the `exdqmlmMCMC()` and `exdqmlmLDVB()` functions.

Predictive scoring

The continuous ranked probability score (CRPS) evaluates the full posterior predictive distribution (Gneiting and Raftery 2007). Let $\rho_p(x) = x[p - I(x < 0)]$ denote the check loss at level p . For a predictive distribution G and observation y , the CRPS is

$$\text{CRPS}(y, G) = \mathbb{E}_{Y \sim G} |Y - y| - \frac{1}{2} \mathbb{E}_{Y, Y' \sim G} |Y - Y'| = 2 \int_0^1 \rho_p(y - G^{-1}(p)) \, dp, \quad (13)$$

which shows that CRPS integrates check loss across all quantile levels and therefore evaluates the full predictive distribution rather than evaluating a single target level p_0 . In `diagnostics()`, posterior predictive draws define empirical quantiles $\hat{q}_t(\tau_k)$, giving the finite approximation

$$\widehat{\text{CRPS}}_t = 2 \sum_{k=1}^K w_k \rho_{\tau_k} \{y_t^{\text{obs}} - \hat{q}_t(\tau_k)\}, \quad (14)$$

where the weights w_k sum to one (Laio and Tamea 2007). The package uses the grid $\tau_k = 0.01, 0.02, \dots, 0.99$ with equal weights and reports the CRPS values in (14) averaged over time. Smaller CRPS values indicate better calibrated models and sharper predictive performance.

For a deterministic forecast, CRPS reduces to the absolute error, so its sample mean CRPS equals the mean absolute error (MAE).

Motivated by the posterior-predictive loss (PPL) framework of [Gelfand and Ghosh \(1998\)](#), the package returns `pplc`, which evaluates the posterior predictive distribution under the target-level check loss ρ_{p_0} . That is,

$$\text{PPLC} = \sum_t \mathbb{E}[\rho_{p_0}(y_t^{\text{obs}} - y_t^{\text{rep}}) \mid y_{1:T}], \quad (15)$$

where the expectation is with respect to posterior predictive replicate draws generated from the observation equation (1) under posterior draws of the model parameters. Smaller PPLC values indicate better performance.

Samples from the posterior replicate distributions are included in the output of `exdqlmMCMC()` and `exdqlmLDVB()`.

Then, the function `diagnostics()` computes these calibration and predictive summaries for fitted dynamic objects. It returns an `exdqlmDiagnostic` object that can be inspected with `print()` or `summary()` and visualized with `plot()`.

k-step-ahead forecasting

For an arbitrary time $\tilde{t}$, the k -step-ahead future marginal distribution of the quantile is

$$\mathbf{F}_{\tilde{t}+k}^\top \boldsymbol{\theta}_{\tilde{t}+k} \mid y_1, \dots, y_{\tilde{t}} \sim \mathcal{N}(\mathbf{F}_{\tilde{t}+k}^\top \mathbf{a}_{\tilde{t}}(k), \mathbf{F}_{\tilde{t}+k}^\top \mathbf{R}_{\tilde{t}}(k) \mathbf{F}_{\tilde{t}+k}) \quad (16)$$

where $\mathbf{a}_{\tilde{t}}(k) = \mathbf{G}_{\tilde{t}+k} \mathbf{a}_{\tilde{t}}(k-1)$, $\mathbf{R}_{\tilde{t}}(k) = \mathbf{G}_{\tilde{t}+k} \mathbf{R}_{\tilde{t}}(k-1) \mathbf{G}_{\tilde{t}+k}^\top + \mathbf{W}_{\tilde{t}+k}$, $\mathbf{a}_{\tilde{t}}(0) = \mathbf{m}_{\tilde{t}}$, and $\mathbf{R}_{\tilde{t}}(0) = \mathbf{C}_{\tilde{t}}$, with $\mathbf{m}_{\tilde{t}}$ and $\mathbf{C}_{\tilde{t}}$ denoting the filtered mean and covariance of $\boldsymbol{\theta}_{\tilde{t}}$, respectively. This distribution is implemented in the standard `predict()` method for fitted dynamic objects and is available for both MCMC and LDVB fits. The fitted object supplies the filtered posterior summaries at the forecast origin, while optional future observation and evolution matrices are supplied when the required forecast design is not already stored in the fitted model. Table 5 summarizes the forecast inputs and optional posterior predictive draw controls. The returned `exdqlmForecast` object contains the forecast-quantile summaries and state moments, with posterior-predictive draws included when requested. Forecast objects can then be plotted or evaluated against held-out observations using `diagnostics()`.

3. Package design and implementation

The package interface follows three main design principles. First, model specification is separated from estimation so that the same state-space model can be used with different inference engines. Second, engine-specific S3 classes are combined with shared family classes to provide common behavior without breaking existing code. Third, post-processing operations return explicit reusable objects, separating computation from printing, summarization, and visualization. Table 6 summarizes the resulting workflow.

Model specification functions such as `polytrendMod()`, `seasMod()`, and `regMod()` return objects of class `exdqlm`. These objects store the state-space matrices, initial-state prior, and component structure used by the fitting routines, and they can be combined with the `+` method to build larger dynamic models from simpler components. This structure allows

Quantity	Mathematical symbol	R argument	Input/default
Fitted dynamic model	$(\mathbf{m}_{\tilde{t}}, \mathbf{C}_{\tilde{t}})$	object	Required fitted dynamic object from <code>exdqlmMCMC()</code> , <code>exdqlmLDVB()</code> , or legacy <code>exdqlmISVB()</code> .
Forecast origin	$\tilde{t}$	start.t	Required integer time index within the fitted model.
Forecast horizon	k	k	Required positive integer number of forecast steps.
Future observation vectors	$\mathbf{F}_{(\tilde{t}+1):(\tilde{t}+k)}$	fFF	Optional $q \times 1$ or $q \times k$ matrix; required with fGG when forecasting beyond the stored model matrices.
Future evolution matrices	$\mathbf{G}_{(\tilde{t}+1):(\tilde{t}+k)}$	fGG	Optional $q \times q$ matrix or $q \times q \times k$ array; required with fFF when forecasting beyond the stored model matrices.
Credible interval mass	$1 - \alpha$	cr.percent	Optional scalar in $(0, 1)$; default 0.95 .
Forecast draws	$y_{\tilde{t}+1:\tilde{t}+k}^{rep}$	return.draws; n.samp; seed	Optional controls; default return.draws = FALSE . When requested, n.samp sets the number of draw columns and seed makes generation reproducible.

Table 5: Forecast notation and principal arguments to the `predict()` method for fitted dynamic objects. Here q denotes the state dimension.

a single model object to be used with different inference engines, AL/DQLM settings, or transfer-function extensions.

Fitting routines return list-based S3 objects whose class vectors include an engine-specific first class and a shared family class. Dynamic fits keep `exdqlmLDVB`, `exdqlmMCMC`, or `exdqlmISVB` as their engine-specific class and also inherit from the shared `exdqlmFit` family. Static fits keep `exalStaticLDVB` or `exalStaticMCMC` as their engine-specific class and inherit from `exalStaticFit`. The shared family classes provide common behavior while preserving the engine-specific class names used by existing code. Fitted objects can be inspected with `print()` and `summary()`, and dynamic fits support `plot()` for fitted quantiles, component contributions, and state summaries.

Post-processing uses S3 methods when an operation is defined on an existing fitted or forecast object. Thus, `predict()` and `diagnostics()` dispatch on the class of their input, while `print()`, `summary()`, and `plot()` operate on the objects they return. Named constructors are used for operations that combine several objects or draw matrices rather than acting on a single object. In particular, `quantileSynthesis()` remains a named function because it combines posterior-predictive information from several separately fitted quantile levels. Named helper functions are retained where needed for backward compatibility, but the method-based interface is the preferred workflow for new code.

For dynamic fit objects in the `exdqlmFit` family, `predict()` returns an `exdqlmForecast` object and `diagnostics()` returns an `exdqlmDiagnostic` object. Forecast objects store forecast quantiles and, when requested, posterior predictive forecast draws. They can be inspected with `print()`, summarized with `summary()`, plotted with `plot()`, and scored against held-out observations with `diagnostics()`. The latter method returns an `exdqlmForecastDiagnostic` object. Dynamic diagnostic objects contain calibration and predictive diagnostics for the fit-

ted model described above and can be inspected with `print()`, summarized with `summary()`, and visualized with `plot()`.

The function `quantileSynthesis()` combines posterior predictive draws from separately fitted quantiles into a single synthesized posterior predictive distribution and returns an `exdqlmSynthesis` object. Static fitted objects in the `exalStaticFit` family support fitted-quantile plots through `plot()` and static model assessment through `diagnostics()`. The latter returns an `exalStaticDiagnostic` object with fitted-quantile summaries, optional holdout or reference metrics, and coefficient summaries that can be printed, summarized, or plotted.

The object system also provides several extension points. Compatible user-supplied state-space specifications can be converted to `exdqlm` objects through `as.exdqlm()` and combined with existing model components through the `+` method. For developers, the shared fitted-object families provide a pattern for incorporating additional inference engines: an engine-specific class can also include `exdqlmFit` or `exalStaticFit` in its class vector and thereby use the common methods, with specialized methods supplied only when its behavior differs. Additional displays and post-processing procedures can similarly be implemented as S3 methods for the fitted, forecast, diagnostic, or synthesis object families.

Computationally intensive operations have C++ implementations through **Rcpp**, with R implementations retained for numerical parity checks and portability. These include the exported exAL density, distribution, quantile, and random-generation utilities; selected Kalman recursions and model builders; posterior and latent-variable sampling paths; posterior predictive simulation; and MCMC backend routing. Package options control backend routing for Kalman recursions, model builders, posterior sampling, posterior predictive simulation, and MCMC routing. In version 1.1.1, compiled stochastic helper paths used by the manuscript run through serial R-controlled random-number streams, rather than worker-thread or private C++ random-number streams.

The variational routines return standardized traces for the ELBO, scale, skewness, and convergence diagnostics, with the closed-form, structured scale-skewness, and legacy Laplace-delta ELBO components documented in the appendices. The examples use fixed benchmark settings, recorded random-number seeds, and a documented reference environment so that run times and stochastic outputs are reproducible.

Computational details and replication. The manuscript results were generated with R 4.6.0 and `exdqlm` 1.1.1 on x86_64 Linux. The replication script sets `RNGversion("4.6.0")` and `RNGkind("Mersenne-Twister", "Inversion", "Rejection")` before applying the recorded seeds. Native thread variables for OpenMP and common BLAS implementations are set to one before launching R; the benchmark settings used in the examples enable the "fast" C++ MCMC backend and keep stochastic compiled helper streams controlled by the R random-number generator. The submitted replication archive contains a flat `code.R` script intended to be run with R CMD BATCH `-vanilla code.R code.Rout`. That run prints fitted-object output, Tables 7–10, selected scalar summaries, and `sessionInfo()` to `code.Rout`, and writes the manuscript figures to `Rplots.pdf` and to the PNG files used by this article. Runtime entries in the tables are elapsed fitting times on the recorded reference platform and are not intended to be machine-independent constants. These computational controls affect implementation and performance rather than the statistical models being fitted.

Function or method	Returned class or output	Role in the workflow
<i>Dynamic model construction</i>		
<code>polytrendMod()</code>	<code>exdqlm</code>	Create a polynomial-trend component.
<code>seasMod()</code>	<code>exdqlm</code>	Create a Fourier-form seasonal or periodic component.
<code>regMod()</code>	<code>exdqlm</code>	Create a regression component from covariates.
<code>as.exdqlm()</code>	<code>exdqlm</code>	Coerce a compatible list or time-invariant <code>d1m</code> object.
<i>Dynamic model fitting – Input <code>exdqlm</code> objects</i>		
<code>exdqlmMCMC()</code>	<code>exdqlmMCMC</code> , <code>exdqlmFit</code>	Fit a dynamic exDQLM or DQLM using MCMC.
<code>exdqlmLDVB()</code>	<code>exdqlmLDVB</code> , <code>exdqlmFit</code>	Fit a dynamic exDQLM or DQLM using LDVB.
<code>exdqlmTransferLDVB()</code>	<code>exdqlmLDVB</code> , <code>exdqlmFit</code>	Fit a transfer-function exDQLM using LDVB.
<code>exdqlmTransferMCMC()</code>	<code>exdqlmMCMC</code> , <code>exdqlmFit</code>	Fit a transfer-function exDQLM using MCMC.
<i>Static model fitting – Input design matrix and response</i>		
<code>exalStaticLDVB()</code>	<code>exalStaticLDVB</code> , <code>exalStaticFit</code>	Fit static exAL or AL regression using LDVB.
<code>exalStaticMCMC()</code>	<code>exalStaticMCMC</code> , <code>exalStaticFit</code>	Fit static exAL or AL regression using MCMC.
<i>Dynamic fit examination – Input <code>exdqlmFit</code> objects</i>		
<code>print()</code> / <code>summary()</code>	Printed or structured summary	Inspect the class, engine, sample size, state dimension, stored draws, convergence, and run time.
<code>plot()</code>	Graphical display	Plot fitted dynamic quantiles, component contributions, or state summaries.
<code>predict()</code>	<code>exdqlmForecast</code>	Compute forecast quantiles and optional posterior-predictive draws.
<code>diagnostics()</code>	<code>exdqlmDiagnostic</code>	Compute calibration and predictive diagnostics; use <code>plot()</code> to visualize them.
<i>Forecast examination – Input <code>exdqlmForecast</code> objects</i>		
<code>print()</code> / <code>summary()</code> / <code>plot()</code>	Summary or graphical display	Inspect and visualize forecast-quantile summaries and posterior-predictive intervals.
<code>diagnostics()</code>	<code>exdqlmForecastDiagnostic</code>	Score forecasts against held-out observations.
<i>Static fit examination – Input <code>exalStaticFit</code> objects</i>		
<code>print()</code> / <code>summary()</code>	Printed or structured summary	Inspect the class, engine, sample size, design dimension, stored draws, and run time.
<code>plot()</code>	Graphical display	Plot fitted conditional-quantile summaries from static LDVB or MCMC fits.
<code>diagnostics()</code>	<code>exalStaticDiagnostic</code>	Compute fitted-quantile diagnostics and coefficient summaries; use <code>plot()</code> for quantile or coefficient displays.
<i>Predictive synthesis – Input dynamic fit or forecast objects, or draw matrices</i>		
<code>quantileSynthesis()</code>	<code>exdqlmSynthesis</code>	Synthesize posterior-predictive draws across separately fitted quantile levels; use <code>print()</code> , <code>summary()</code> , or <code>plot()</code> to examine the result.

Table 6: Core object families, workflow functions, and standard methods used in the examples. For fitted models, the two classes shown in the second column are elements of the object’s S3 class vector: the engine-specific class is retained for backward compatibility, while `exdqlmFit` and `exalStaticFit` provide shared behavior for dynamic and static fits, respectively. Post-processing operations return explicit objects that can be printed, summarized, or plotted where a corresponding method is defined.

4. Examples

We demonstrate the general workflow and key features of the package through three real-data analyses and one simulation benchmark. The Lake Huron analysis illustrates dynamic model construction, MCMC inference, forecasting, and posterior-predictive synthesis across separately fitted quantiles. The sunspot analysis demonstrates model-component composition, LDVB inference, fitted-model diagnostics, discount-factor comparison, and comparison with MCMC. The Big Tree water-flow analysis compares specifications without covariates, with direct regression effects, and with transfer-function effects, including held-out forecasting. Finally, the simulation example illustrates static exAL regression with shrinkage priors and compares LDVB with MCMC.

The package also exposes distribution-level utilities for the exAL error family: `dexal()`, `pexal()`, `qexal()`, and `rexal()` evaluate the density, distribution function, quantile function, and random generator, respectively, while `get_gamma_bounds()` returns the admissible interval for γ at a specified p_0 . These utilities are useful for simulation, prior calibration, and direct checks of the exAL distribution, but are not needed for the main fitting, forecasting, diagnostic, or synthesis workflows illustrated below. We use CrI as shorthand for credible interval in figure captions and plotting labels.

We begin by loading the **exdqlm** package used for all examples.

```
R> library("exdqlm")
```

4.1. Lake Huron

In this example, we consider the Lake Huron time series from the package **datasets**. The data are annual measurements of the level (ft) of Lake Huron from 1875 to 1972, shown in Figure 2. This example shows how to build a basic state-space structure, specify a prior on γ , run the MCMC algorithm, and plot estimated quantiles and forecast distributions.

To estimate the dynamic distribution of the data, we consider three quantiles, $p_0 = 0.95, 0.50$, and 0.05 . We begin by creating the state-space model structure and the prior parameters (i.e. $\mathbf{F}_t, \mathbf{G}_t, \mathbf{m}_0$, and $\mathbf{C}_0$ discussed in Section 2). We choose to model the quantiles with a second order polynomial trend, which we construct with the `polytrendMod()` function. The initial level prior is centered at 579 ft, a rounded domain-scale value chosen before fitting rather than the sample mean, and the initial slope prior is centered at zero.

```
R> model = polytrendMod(order = 2, m0 = c(579, 0), C0 = 10 * diag(2))
```

As an illustration we fix a single discount factor of 0.9 to define the evolution covariance matrix $\mathbf{W}_t$. Discount factors can also be selected using in-sample diagnostic criteria, as illustrated in Section 4.2. The prior location parameters for γ are quantile-specific: they are set to zero for the median and shifted in opposite directions for the upper and lower tails to provide tail-specific regularization within the admissible support. In these fits, the scale parameter is 0.1 and the number of degrees of freedom is 1. These quantities specify prior information and do not determine the MCMC starting values. In the fits shown here we use `n.burn = 2000` and `n.mcmc = 3000`. The computation uses the backend settings described in Section 3; backend and runtime provenance for the manuscript run are recorded in the replication logs.

```

R> set.seed(20260501)
R> M95 = exdqlmMCMC(y = LakeHuron, p0 = 0.95, model = model, df = 0.9, dim.df = 2,
+                 PriorGamma = list(m_gam = -1, s_gam = 0.1, df_gam = 1),
+                 n.burn = 2000, n.mcmc = 3000, verbose = FALSE)
R> M5 = exdqlmMCMC(y = LakeHuron, p0 = 0.05, model = model, df = 0.9, dim.df = 2,
+                 PriorGamma = list(m_gam = 1, s_gam = 0.1, df_gam = 1),
+                 n.burn = 2000, n.mcmc = 3000, verbose = FALSE)

```

The fitted object output is intentionally compact. It identifies the model family, inference engine, quantile level, state dimension, retained posterior draws, and the standard methods available for downstream analysis. The `summary()` method adds draw dimensions and scalar posterior summaries.

```
R> M95
```

```

Dynamic quantile state-space fit
Model: exDQLM (exAL)
Inference engine: MCMC
Quantile level (p0): 0.95
Observations: 98
State dimension: 2
Discount factors (dimensions): 0.9 ( 2 )
Burn-in: 2000
Posterior draws: 3000
State draws: 2 x 98 x 3000
Posterior predictive draws: 98 x 3000
Run-time: 78.187 seconds

Class: "exdqlmMCMC" "exdqlmFit"
Use with: summary(), plot(), predict(), diagnostics()
Plot types: quantile, component, state

```

```
R> summary(M95)
```

```

Dynamic quantile state-space fit
Model: exDQLM (exAL)
Inference engine: MCMC
Quantile level (p0): 0.95
Observations: 98
State dimension: 2
Discount factors (dimensions): 0.9 ( 2 )
Burn-in: 2000
Posterior draws: 3000
State draws: 2 x 98 x 3000
Posterior predictive draws: 98 x 3000
Run-time: 78.187 seconds

Class: "exdqlmMCMC" "exdqlmFit"
Use with: summary(), plot(), predict(), diagnostics()
Plot types: quantile, component, state

```

Stored draws:

	Quantity	Dimension
state draws	2 x 98 x 3000	
posterior predictive draws	98 x 3000	

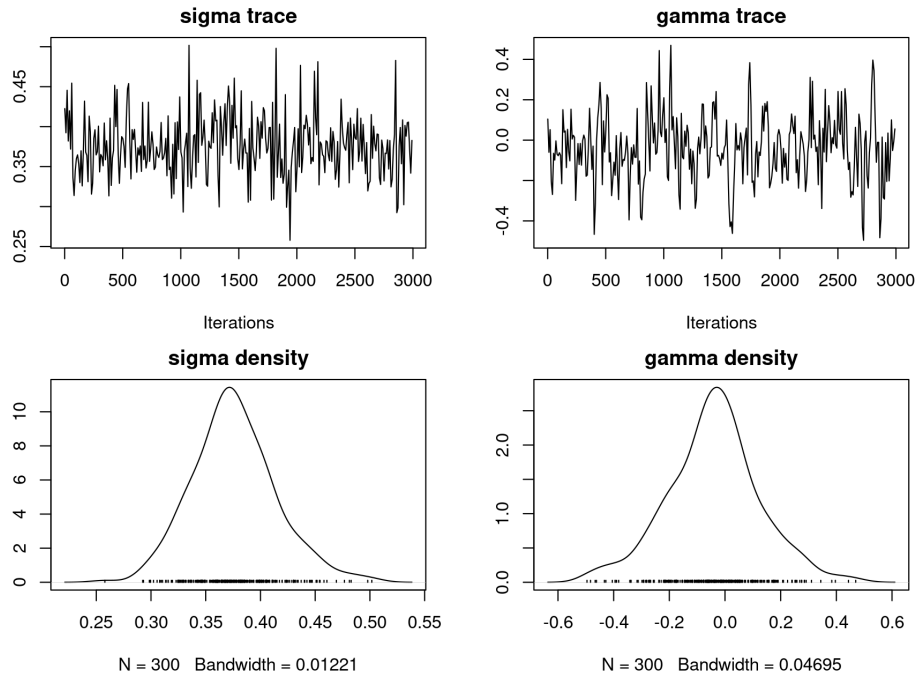


Figure 1: Example 1: Lake Huron. Trace and density plots from the median run with 7000 burn-in iterations and 3000 retained draws. For readability, every 10th retained draw is used in the plotting step. The top row shows the posterior trace plots for σ and γ , and the bottom row shows the corresponding posterior densities. The displayed paths are those produced by the recorded reference run.

```
sigma draws      3000 x 1
gamma draws      3000 x 1
```

Scalar posterior summaries:

Parameter	Mean	SD
sigma	0.2003	0.02553
gamma	-4.5720	0.93906

By default, `exdq1mMCMC()` uses the package's LDVB routine, `exdq1mLDVB()`, to warm-start the chain. This initialization choice is separate from the subsequent MCMC kernel. Because the data are relatively symmetric, we do not expect the skewness parameter γ to be far from zero at $p_0 = 0.5$. For the diagnostic display, we run the unrestricted median model separately as `M50.trace` with a longer burn-in. Consistent with the symmetry of the data, the posterior mean of γ is -0.039 with a 95% credible interval $(-0.397, 0.291)$, while the posterior mean of σ is 0.375 with a 95% credible interval $(0.302, 0.460)$. All retained samples in the output of `exdq1mMCMC()` are `mcmc` objects, so we can use the package `coda` (Plummer, Best, Cowles, and Vines 2006) to examine the trace and density plots of both σ and γ . For readability, we plot every 10th retained draw in Figure 1.

```
R> set.seed(20260616)
R> M50.trace = exdq1mMCMC(y = LakeHuron, p0 = 0.50, model = model, df = 0.9, dim.df = 2,
+                       PriorGamma = list(m_gam = 0, s_gam = 0.1, df_gam = 1),
+                       init.from.vb = TRUE,
```

```

+           vb_init_controls = list(method = "ldvb", verbose = FALSE),
+           mh.proposal = "slice",
+           n.burn = 7000, n.mcmc = 3000, verbose = FALSE)
R> par(mfcol = c(2, 2), mar = c(4.1, 4.1, 2.1, 1.0))
R> keep.idx = seq(1, length(M50.trace$samp.sigma), by = 10)
R> sigma.trace = coda::mcmc(M50.trace$samp.sigma[keep.idx], thin = 10)
R> gamma.trace = coda::mcmc(M50.trace$samp.gamma[keep.idx], thin = 10)
R> coda::traceplot(sigma.trace, main = "sigma trace")
R> coda::densplot(sigma.trace, main = "sigma density")
R> coda::traceplot(gamma.trace, main = "gamma trace")
R> coda::densplot(gamma.trace, main = "gamma density")

```

The unrestricted median run is used only to assess whether the additional skewness parameter is needed at $p_0 = 0.50$. We use a longer burn-in for this diagnostic run because γ is sampled in that fit. After the posterior for γ is found to be concentrated near zero, the median model used in the quantile, forecast, and synthesis panels is the fixed- γ DQLM `M50.dqlm`. Because γ is fixed in this model, the skewness update is removed from the MCMC kernel and we use the same 2000 burn-in iterations and 3000 retained draws used for the other main fits. We re-run the model with a point-mass prior on γ at 0 using the settings `gam.init = 0` and `fix.gamma = TRUE`, which is equivalent to the setting `dqlm.ind = TRUE`. This reduces the model to the DQLM mentioned in Section 2.1.6.

```

R> M50.dqlm = exdqlmMCMC(y = LakeHuron, p0 = 0.50, model = model, df = 0.9, dim.df = 2,
+                       gam.init = 0, fix.gamma = TRUE, n.burn = 2000, n.mcmc = 3000, verbose = FALSE)

```

The results are `exdqlmMCMC` objects that can be used for analysis and forecasting. First we examine the estimated quantiles with the plot method for `exdqlmMCMC` objects, seen in the top-left panel of Figure 2. This generates a plot of the data with posterior mean estimates and 95% CrIs of the dynamic quantile. Subsequent quantiles (i.e. $p_0 = 0.50, 0.05$ in this example) can be added to the same axes by calling `plot()` on the fitted objects with `add = TRUE`.

```

R> par(mfrow = c(2, 2), mar = c(4.4, 4.1, 2.2, 1.2), oma = c(0, 0, 0.8, 0))
R> plot(M95); title("(a) Dynamic quantiles")
R> plot(M50.dqlm, add = TRUE, col = "blue")
R> plot(M5, add = TRUE, col = "forest green")
R> legend("topright", lty = 1, bty = "n", col = c("purple", "blue", "forest green"),
+       legend = c(expression('p'[0]*'=0.95'), expression('p'[0]*'=0.50'),
+       expression('p'[0]*'=0.05')))

```

Next, we forecast the $k = 8$ step-ahead distributions starting in 1972. To do this, we first create the observational vector (`fFF`) and evolution matrix (`fGG`) that will be used in the forecast updates. In this case both are non-time-varying and identical to those used to estimate the quantile.

```

R> fFF = model$FF
R> fGG = model$GG

```

We plot the time series data in a narrower range for closer examination. Forecast objects can be created with `predict()` on a fitted dynamic model. Here we create the forecast objects first and then call `plot()` with `add = TRUE`. Notice we start the forecast at the last time index of the data, i.e. `start.t = length(LakeHuron)`.

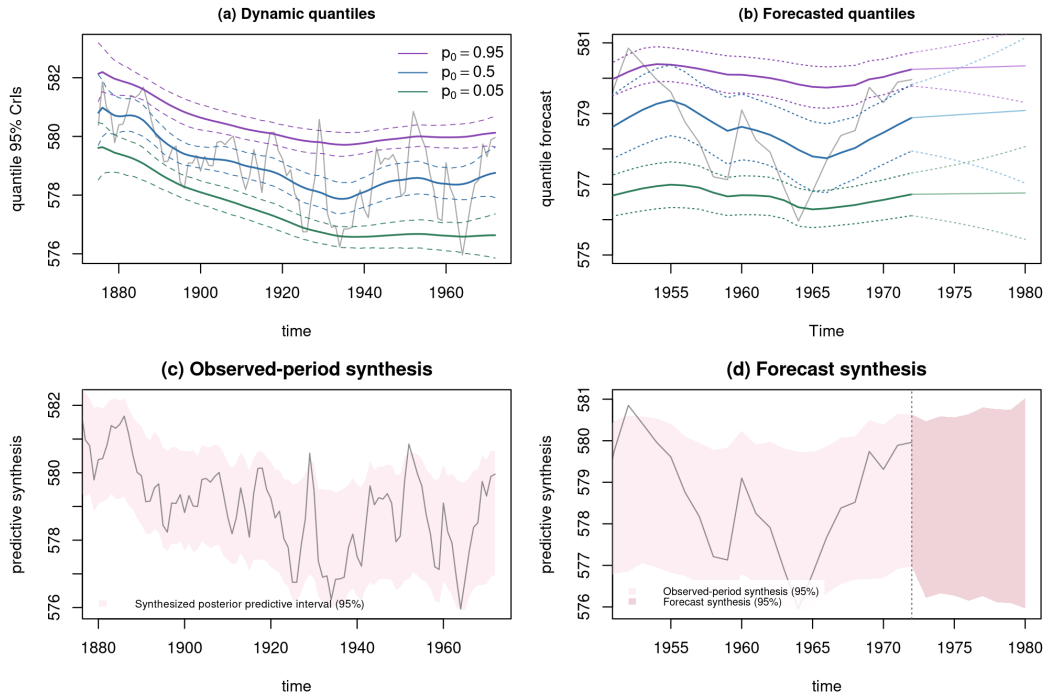


Figure 2: Example 1: Lake Huron. Top-left: posterior mean estimates and 95% CrIs of the estimated quantiles, plotted with the data (grey). Top-right: forecasted quantile estimates and 95% credible intervals (seen after 1972), along with the filtered quantile estimates and 95% credible intervals (seen from 1952 to 1972). Bottom-left: synthesized posterior predictive 95% interval over the observed period. Bottom-right: observed-period synthesis near the forecast origin (lighter band) and synthesized posterior predictive 95% interval over the eight-step forecast window (slightly darker rose band); the vertical dotted line marks the forecast origin.

```
R> plot(LakeHuron, xlim = c(1952, 1980), ylim = c(575, 582), col = "dark grey",
+       main = "(b) Forecasted quantiles", ylab = "forecast 95% CrIs")
R> fc95 = predict(M95, start.t = length(LakeHuron), k = 8,
+               fFF = fFF, fGG = fGG, return.draws = TRUE)
R> fc50 = predict(M50.dqlm, start.t = length(LakeHuron), k = 8,
+               fFF = fFF, fGG = fGG, return.draws = TRUE)
R> fc05 = predict(M5, start.t = length(LakeHuron), k = 8,
+               fFF = fFF, fGG = fGG, return.draws = TRUE)
R> plot(fc95, add = TRUE)
R> plot(fc50, add = TRUE, cols = c("blue", "light blue"))
R> plot(fc05, add = TRUE, cols = c("forest green", "green"))
```

This generates a plot of the data with the forecasted quantile estimates and 95% credible intervals (seen after 1972), along with the filtered quantile estimates and 95% credible intervals (before 1972), for reference. Results can be seen in the top-right panel of Figure 2. The percentage in the CrIs can be modified with the parameter `cr.percent`.

When several quantile models are fitted separately, the package also allows their posterior predictive draws to be combined into a single coherent predictive distribution through `quantileSynthesis()`. Each fitted model contributes local predictive information near its

target quantile, and the synthesis step interpolates across these quantile-specific draws to obtain one posterior predictive distribution. When independently fitted quantiles induce crossing, optional monotonicity corrections based on isotonic adjustment and monotone rearrangement are available (Barlow, Bartholomew, Bremner, and Brunk 1972; Chernozhukov, Fernández-Val, and Galichon 2010). The synthesis operates on posterior predictive draws indexed by quantile level, so the input fits need not use identical skewness restrictions. For the Lake Huron example, we combine exDQLM tail fits at $p_0 = 0.05$ and 0.95 with the fixed- γ DQLM median fit `M50.dqlm`.

```
R> syn.obs = quantileSynthesis( draws_list = list(M5, M50.dqlm, M95), p = c(0.05, 0.50, 0.95),
+                               enforce_isotonic = TRUE, rearrange = TRUE, T_expected = length(LakeHuron))
```

The recorded manuscript run uses the package’s isotonic adjustment and monotone rearrangement; these are the default correction settings and are shown explicitly in the displayed calls. The stored synthesized quantile summaries have no remaining crossings over the observed period.

Because the argument `return.draws = TRUE` was used, each object stores posterior predictive forecast draws in the return value `samp.fore`. This allows us to apply the same synthesis step to the eight-step forecast horizon.

```
R> syn.fore = quantileSynthesis(draws_list = list(fc05, fc50, fc95), p = c(0.05, 0.50, 0.95),
+                               enforce_isotonic = TRUE, rearrange = TRUE, T_expected = 8)
```

The same correction settings are used for the forecast synthesis, and the eight-step synthesized forecast summaries are also ordered across the three input quantile levels.

The objects `syn.obs` and `syn.fore` have class `exdqlmSynthesis`, so the corresponding `plot()` method can be used to display the synthesized posterior predictive intervals. The lower panels of Figure 2 are produced from these objects as follows. The first synthesis panel shows the observed-period synthesized predictive interval, and the second panel uses the same observed-period synthesis near the forecast origin, overlays the forecast synthesis, and shades the one-step bridge from the final observed synthesis interval to the first forecast synthesis interval.

```
R> synth.obs.col = adjustcolor("#F7D6DE", alpha.f = 0.40)
R> synth.fore.col = adjustcolor("#D98A9B", alpha.f = 0.38)
R> plot(syn.obs, y = LakeHuron, time = as.numeric(time(LakeHuron)), xlim = c(1880, 1972),
+       ylim = c(575.75, 582.25), ylab = "predictive synthesis",
+       main = "(c) Observed-period synthesis", show.median = FALSE,
+       band.col = synth.obs.col, y.col = adjustcolor("grey30", alpha.f = 0.62))
R> legend("bottomleft", legend = "Synthesized posterior predictive interval (95%)",
+       fill = synth.obs.col, border = NA, bty = "n", cex = 0.68)
R> plot(syn.obs, y = LakeHuron, time = as.numeric(time(LakeHuron)), xlim = c(1952, 1980),
+       ylim = c(575.75, 581), ylab = "predictive synthesis",
+       main = "(d) Forecast synthesis", show.median = FALSE, band.col = synth.obs.col,
+       y.col = adjustcolor("grey30", alpha.f = 0.62))
R> polygon(c(1972, 1973, 1972), c(tail(syn.obs$summary$q025, 1),
+   syn.fore$summary$q025[1], syn.fore$summary$q975[1],
+   tail(syn.obs$summary$q975, 1))), col = synth.fore.col, border = NA)
R> plot(syn.fore, time = 1973:1980, add = TRUE, show.median = FALSE,
+       band.col = synth.fore.col)
R> abline(v = 1972, lty = 3)
```

```
R> legend("bottomleft", legend = c("Observed-period synthesis (95%)",
+   "Forecast synthesis (95%)"),
+   fill = c(synth.obs.col, synth.fore.col), border = NA, bty = "n", cex = 0.66)
```

4.2. Sunspots

For our next example, we use the yearly Sunspot time series from the package **datasets**. The data are yearly counts for sunspots from 1700 to 1988, shown in Figure 3. In this example we show how to use the **dlm** package to create the state-space model, combine blocks of a state-space model, apply the LDVB approximation through **exdqlmLDVB()**, perform visual model diagnostics to compare the exDQLM with the DQLM, and use those diagnostics for discount-factor selection.

Upper-tail sunspot activity is a natural setting for illustrating dynamic quantile modeling of cyclic count data. We therefore model the 0.85 quantile in this example. Although sunspot activity is recorded as a count, we use the continuous exAL working likelihood to model its upper conditional quantile; the example is not intended as a discrete-count likelihood analysis. Again, we begin by building the state-space structure used to estimate the quantile. Here we choose to model the quantile with a first order polynomial trend component as well as a Fourier form seasonal component that incorporates a fundamental period every 11 observations (years), as well as some of its corresponding harmonics. This results in a dynamic model with a total of 9 state parameters, 1 for the first order polynomial component and 8 for the seasonal component. The **exdqlm** functions are compatible with time-invariant **dlm** objects from the **dlm** package. For example, we create the first order trend component with the function **dlmModPoly()** from the **dlm** package, then use **as.exdqlm()** to turn the output into an **exdqlm** object.

```
R> dlm.trend.comp = dlm::dlmModPoly(1, m0 = 50, C0 = 2500)
R> trend.comp = as.exdqlm(dlm.trend.comp)
```

The initial level prior is centered at 50 sunspots, a rounded count-scale value chosen before fitting, with prior standard deviation 50 to keep the level weakly informative on the observed count scale. Next, we construct the seasonal component with our **seasMod()** function. Sunspot cycles are commonly modeled with an approximately 11-year period; we include the first four Fourier harmonics in the component.

```
R> seas.comp = seasMod(p = 11, h = 1:4, C0 = 10 * diag(8))
```

To combine the two components into a single state-space structure, we add the two **exdqlm** objects.

```
R> model = trend.comp + seas.comp
```

Adding the two model components produces a block-composed state-space model with one trend state and eight seasonal states.

We will apply discount factors by component, i.e. a discount factor of 0.9 for the trend component of dimension 1 and a discount factor of 0.85 for the seasonal component of dimension 8. This discounting strategy can be applied with arguments **df = c(0.9, 0.85)** and **dim.df = c(1, 8)**.

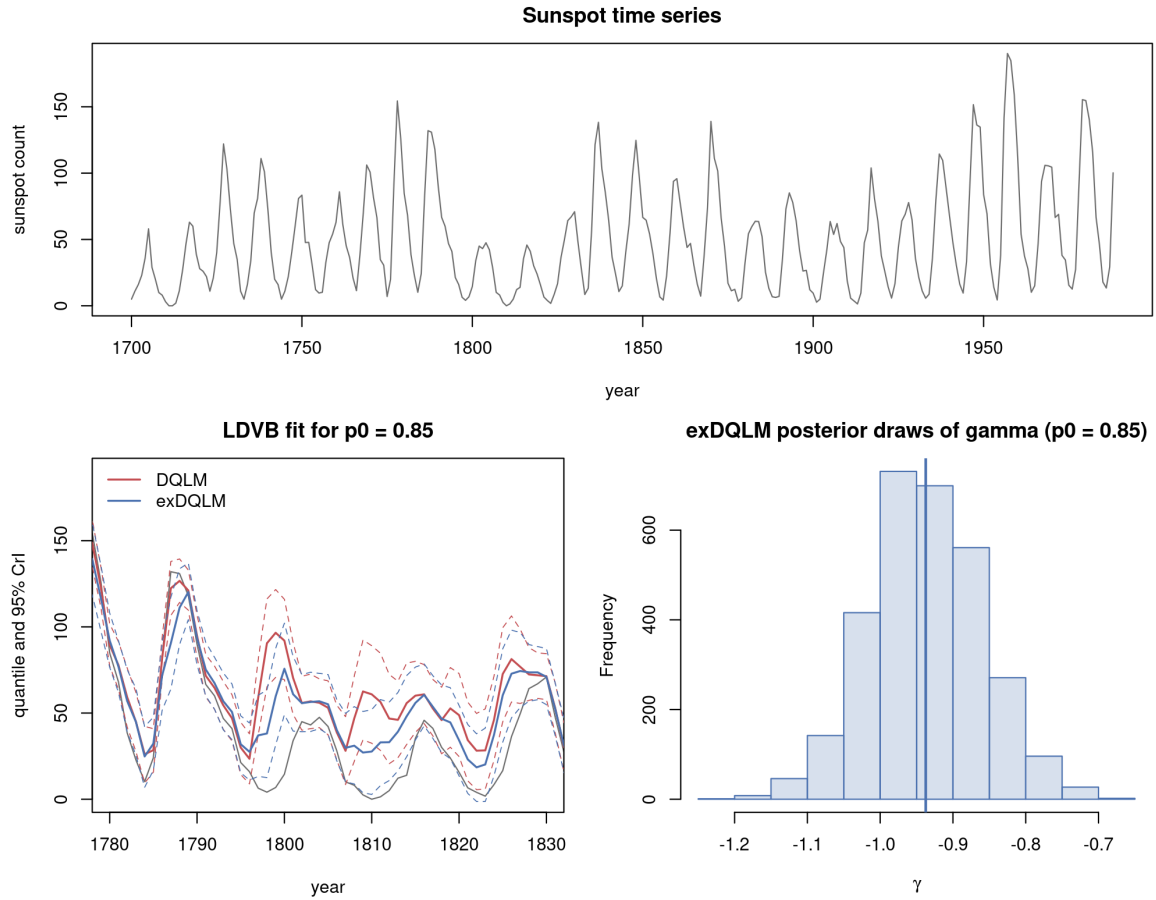


Figure 3: Example 2: Sunspots. Top: The sunspot time series from 1700 to 1988. Bottom left: posterior mean estimates and 95% CrIs of the estimated quantiles from the DQLM (red) and exDQLM (blue), plotted with the data (grey) from 1780 to 1830. Bottom right: Histogram of samples from the approximated posterior distribution of γ under the exDQLM.

We call the function `exdqlmLDVB()` to obtain the LDVB approximations. We use the routine with argument `dqlm.ind = TRUE` to estimate the DQLM, as well as with the default settings to estimate the exDQLM. In this example we set `sig.init = 2` as the starting value for the LDVB scale update, use `n.samp = 3000`, and suppress iteration output with `verbose = FALSE`.

```
R> set.seed(20262601)
R> M1 = exdqlmLDVB(y = sunspot.year, p0 = 0.85, model = model, df = c(0.9, 0.85),
+               dim.df = c(1, 8), dqlm.ind = TRUE, fix.sigma = FALSE,
+               sig.init = 2, n.samp = 3000, verbose = FALSE)
R> set.seed(20262602)
R> M2 = exdqlmLDVB(y = sunspot.year, p0 = 0.85, model = model, df = c(0.9, 0.85),
+               dim.df = c(1, 8), sig.init = 2, fix.sigma = FALSE,
+               n.samp = 3000, verbose = FALSE)
```

The fitted objects can be summarized with the `summary()` generic, which reports the model class, sample size, state dimension, discount factors, convergence iterations, and run time. For

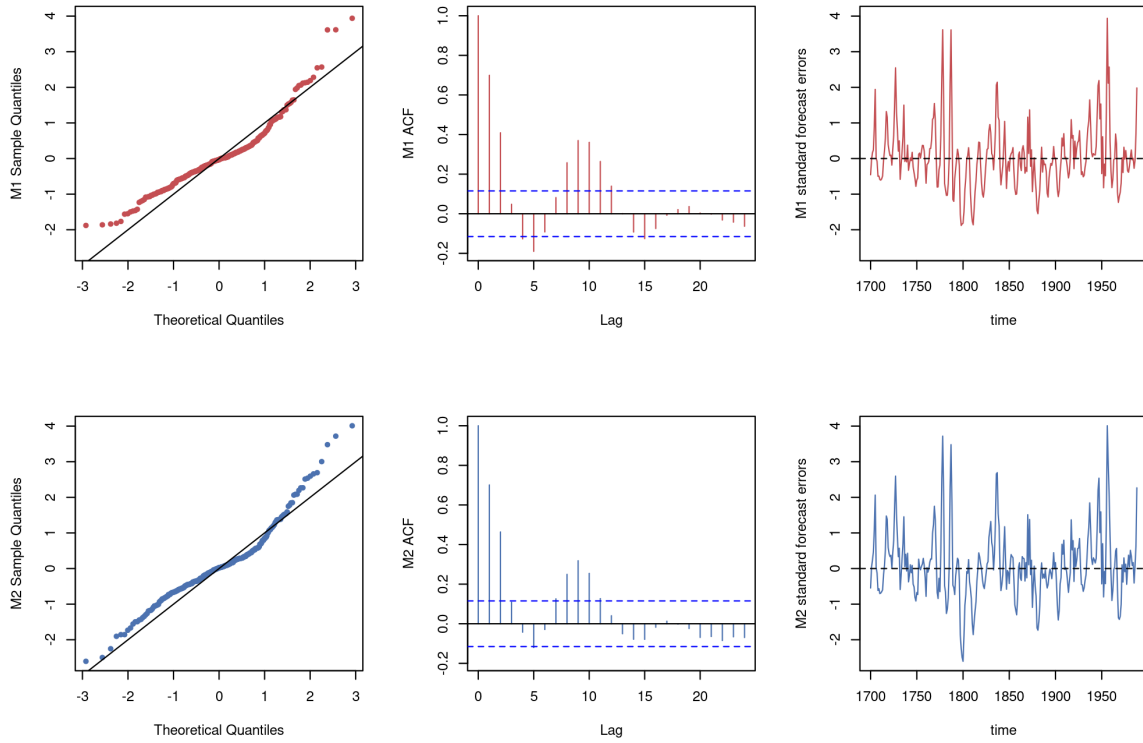


Figure 4: Example 2: Sunspots. The QQ plots (left column), ACF plots of the PIT sequence (center column), and MAP one-step-ahead standardized forecast errors (right column) from the DQLM (red) and exDQLM (blue).

this dataset of length 289, the DQLM LDVB fit completes more quickly, while the corresponding exDQLM LDVB fit is more computationally demanding because the skewness parameter is estimated rather than fixed. The timing comparison in Table 7 reports the benchmark values used in the manuscript under the recorded benchmark settings. These runtimes are fit elapsed times stored in the returned fit objects; they are intended for within-setting comparison and should not be interpreted as machine-independent timing constants.

We can visualize the differences between the two resulting dynamic quantiles with the fitted-object `plot()` method, shown in the lower-left panel of Figure 3. For clarity we limit that panel to years 1780 to 1830.

```
R> plot(sunspot.year, xlim = c(1780, 1830), col = "dark grey", ylab = "quantile 95% CrIs")
R> plot(M1, add = TRUE, col = "red")
R> plot(M2, add = TRUE, col = "blue")
R> legend("topleft", lty = 1, bty = "n", col = c("red", "blue"), legend = c("DQLM", "exDQLM"))
R> title("LDVB fit for  $p_0 = 0.85$ ")
```

To examine whether the added flexibility of the exDQLM is supported by the fitted approximation, we can also visualize the approximate posterior distribution of the skewness parameter γ by plotting the samples from the corresponding variational distribution, also seen in Figure

3. The approximate posterior mass lies away from zero, supporting the additional skewness flexibility for this upper-quantile fit.

```
R> hist(M2$samp.gamma, xlab = expression(gamma), main = "",
+       col = adjustcolor("#4C72B0", alpha.f = 0.22), border = "#4C72B0")
R> abline(v = median(M2$samp.gamma), col = "#4C72B0", lwd = 2)
R> title("exDQLM posterior draws of gamma (p0 = 0.85)")
```

To further examine the two models, we use the `diagnostics()` method to create an object of class `exdqlmDiagnostic` and then call its `plot()` method. The results are seen in Figure 4. These diagnostic plots are based on the MAP one-step-ahead standardized forecast errors produced by the filtering recursion, together with the corresponding probability integral transform (PIT) sequence. Thus the figure summarizes the fitted diagnostic sequence rather than posterior bands for residuals.

```
R> par(mfrow = c(2, 3))
R> diagM1M2 = diagnostics(M1, M2)
R> plot(diagM1M2, cols = c("red", "blue"))
```

The `diagnostics()` method can also be used for a coarse in-sample comparison of candidate model specifications. Here we examine whether the discount factor used for the seasonal component is reasonable under the reported diagnostic criteria. We create a grid of possible discount factors, run the LDVB approximation for each candidate model, and compare the resulting CRPS and KL values. Because these criteria are computed using the observations employed to fit each model, this exercise is a diagnostic screening rather than an out-of-sample predictive validation. We use CRPS as the primary comparison criterion and KL divergence as a complementary calibration check; the selected discount-factor specification is the row with the smallest CRPS.

```
R> possible.dfs = cbind(0.9, seq(0.85, 1, 0.05))
R> possible.dfs

      [,1] [,2]
[1,]  0.9 0.85
[2,]  0.9 0.90
[3,]  0.9 0.95
[4,]  0.9 1.00

R> metrics <- matrix(NA_real_, nrow(possible.dfs), 2, dimnames = list(NULL, c("CRPS", "KL")))
R> for (i in 1:nrow(possible.dfs)) {
+   set.seed(20262700 + i)
+   temp.M2 = exdqlmLDVB(y = sunspot.year, p0 = 0.85, model = model, df = possible.dfs[i, ],
+                        dim.df = c(1, 8), sig.init = 2, fix.sigma = FALSE,
+                        n.samp = 3000, verbose = FALSE)
+   temp.check = diagnostics(temp.M2)
+   metrics[i, ] = c(temp.check$m1.CRPS, temp.check$m1.KL)
+ }
R> df.scan = data.frame(possible.dfs, CRPS = metrics[, "CRPS"], KL = metrics[, "KL"])
R> round(df.scan, 3)
```

```
      X1  X2  CRPS  KL
1 0.9 0.85 6.235 0.120
```

model	method	runtime (s)	KL	CRPS	PPLC
DQLM	LDVB	23.40	0.133	8.525	3851.4
	MCMC	155.64	0.175	7.608	2969.0
exDQLM	LDVB	45.54	0.107	6.558	3015.2
	MCMC	207.00	0.152	5.147	2216.1

Table 7: Example 2: representative dynamic benchmark for the DQLM and exDQLM fits at $p_0 = 0.85$. Each row reports fit runtime together with the deterministic KL normality diagnostic, the mean CRPS from posterior predictive empirical quantiles, and the posterior predictive loss criterion (PPLC). The LDVB rows use `n.samp = 3000`. The MCMC rows use 2000 burn-in iterations and 3000 retained draws. Runtimes are fit elapsed times stored in the returned fit objects and exclude diagnostic calculations, plotting, table construction, and manuscript rendering. They were measured under benchmark setting B on an Intel Xeon E5-4650 CPU with R 4.6.0, **exdqlm** version 1.1.1, the C++ MCMC backend in "fast" mode, and `exdqlm.cpp_threads = 1L`. Because the columns are the comparison metrics, bold entries indicate the smaller value within each model block; displayed ties are left unbolded.

```
2 0.9 0.90 8.261 0.145
3 0.9 0.95 12.458 0.173
4 0.9 1.00 20.233 0.256
```

```
R> possible.dfs[which.min(metrics[, "CRPS"]), ]
```

```
[1] 0.90 0.85
```

On this coarse grid, the CRPS and KL criteria agree and both select the seasonal discount factor 0.85, so the choice used above is retained.

To compare the fast variational approximation with posterior simulation, we fit matched DQLM and exDQLM specifications with `exdqlmMCMC()`. The MCMC fits use the same state-space model, quantile level, and discount factors as the LDVB fits, with 2000 burn-in iterations and 3000 retained draws.

```
R> set.seed(20262801)
R> M1mcmc = exdqlmMCMC(y = sunspot.year, p0 = 0.85, model = model, df = c(0.9, 0.85),
+                      dim.df = c(1, 8), n.burn = 2000, n.mcmc = 3000, verbose = FALSE,
+                      dqlm.ind = TRUE, fix.sigma = FALSE)
R> set.seed(20262802)
R> M2mcmc = exdqlmMCMC(y = sunspot.year, p0 = 0.85, model = model, df = c(0.9, 0.85),
+                      dim.df = c(1, 8), n.burn = 2000, n.mcmc = 3000, verbose = FALSE,
+                      fix.sigma = FALSE)
```

Table 7 gives a compact runtime-and-diagnostic comparison. Within each model class, LDVB is substantially faster. For both DQLM and exDQLM, LDVB gives the smaller KL value, while MCMC gives smaller CRPS and PPLC values. Comparing model classes within each inferential method, the exDQLM improves the LDVB diagnostics relative to the DQLM, and under MCMC it improves the predictive-score criteria CRPS and PPLC. The comparison shows why runtime is reported together with predictive criteria rather than by itself.

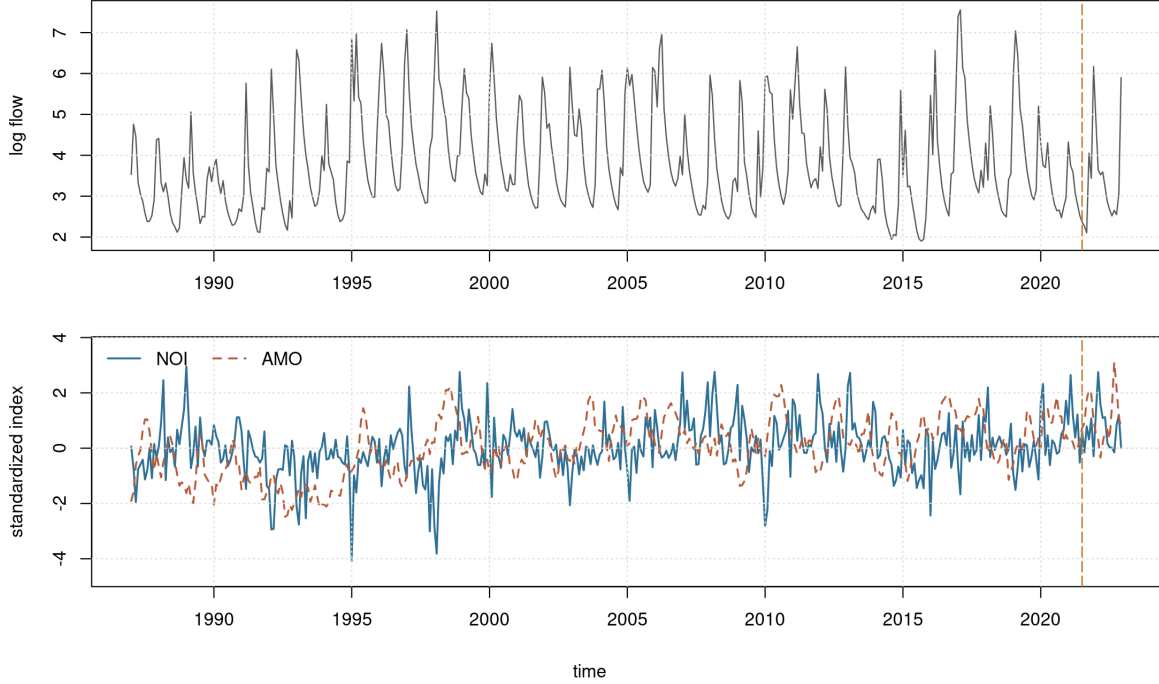


Figure 5: Example 3: Big Tree water flow. Top: observed average monthly water flow at the Big Tree gauge of the San Lorenzo River in Santa Cruz County, CA, plotted on the log scale. Bottom: standardized monthly values of the Northern Oscillation Index (NOI) and Atlantic Multidecadal Oscillation (AMO) index. The displayed modeling window spans January 1987 through December 2022, and the vertical dashed line marks the beginning of the held-out forecast window in July 2021.

4.3. Big Tree water flow

For the third example, we use observed average monthly water flow (cubic feet per second) at the Big Tree gauge of the San Lorenzo River in Santa Cruz County, CA ([U.S. Geological Survey 2016](#)). The **exdqlm** version 1.1.1 used for this article includes these measurements as the monthly time series `BTflow`, which extends through March 2026. Here we use the complete overlap between `BTflow` and the package data frame `climateIndices`, giving 432 monthly observations from January 1987 through December 2022. The final 18 months, July 2021 through December 2022, are not used for fitting and are shown later in a conditional forecast display. This example illustrates the transfer-function workflow: building a baseline dynamic quantile model, adding external covariates through the transfer-function state augmentation, examining fitted components, and forecasting conditionally on future covariate values.

We focus on the lower-tail quantile $p_0 = 0.15$, which represents lower monthly-flow behavior on the log scale. As transfer-function inputs, we use two monthly climate indices from `climateIndices`: the Northern Oscillation Index (NOI) and the Atlantic Multidecadal Oscillation (AMO) index. These indices come from the NOAA Physical Sciences Laboratory climate-index collection ([NOAA Physical Sciences Laboratory 2026](#)) and provide a compact setting for illustrating external-input effects. The covariates are standardized using the training window only. In the forecast display, NOI and AMO values over the held-out months are

treated as known future inputs; the display is therefore conditional on those covariates rather than an operational forecast of the climate indices. The complete overlap used here has 432 monthly observations. The first 414 observations, January 1987 through June 2021, form the training period; the final 18 observations, July 2021 through December 2022, form the forecast holdout. The following chunk shows the essential data alignment. The replication script adds date and completeness checks, uses the same training-window scaling, and writes the aligned modeling data to `analysis/manuscript/outputs/tables/ex3_model_dataset.csv`.

```
R> data("BTflow", package = "exdqlm")
R> data("climateIndices", package = "exdqlm")
R> ex3.dates = seq(as.Date("1987-01-01"), by = "month", length.out = length(BTflow))
R> flow.df = data.frame(date = ex3.dates, flow = as.numeric(BTflow))
R> ex3.df = merge(flow.df, climateIndices[, c("date", "noi", "amo")], by = "date")
R> ex3.df = ex3.df[1:432, ]
R> y.fit = ts(log(ex3.df$flow), start = c(1987, 1), frequency = 12)
R> y.train = window(y.fit, end = c(2021, 6))
R> y.holdout = window(y.fit, start = c(2021, 7))
R> X.raw = as.matrix(ex3.df[, c("noi", "amo")])
R> X.train = scale(X.raw[1:414, ])
R> X.holdout = scale(X.raw[415:432, ], center = attr(X.train, "scaled:center"),
+                  scale = attr(X.train, "scaled:scale"))
```

The baseline state-space model contains a first-order polynomial trend and a Fourier-form seasonal component with period 12. The seasonal block includes $h = 1$, $h = 2$, and $h \approx 0.147$, corresponding to annual, semiannual, and approximately 7-year cycles on the monthly scale. The low-frequency harmonic $h = 0.1469118636$ follows the specification developed in [Barata et al. \(2022\)](#); see that work for its motivation and selection.

```
R> trend.comp = polytrendMod(1, m0 = log(50), C0 = 1)
R> seas.comp = seasMod(p = 12, h = c(1, 2, 0.1469118636), C0 = diag(1, 6))
R> model = trend.comp + seas.comp
```

The initial log-flow level prior is centered at $\log(50)$, corresponding to a fixed 50 cubic feet per second reference value, with prior standard deviation 1 on the log scale. This gives a broad domain-scale prior without using the training response to center the state prior. The standardized climate coefficients are assigned zero-centered normal priors with variance 1. On the log-flow scale this is intentionally weakly informative: it allows either climate index to shift the lower-tail quantile materially while still regularizing the example around no climate effect.

We compare three models built from the same trend and seasonal baseline. The first, `M0`, contains no climate covariates. The second, `MREG`, adds NOI and AMO as direct contemporaneous regressors using `regMod()`. The third, `MTF`, adds the same two covariates through the transfer-function structure from Section 2.1.5. For all three models, the trend and seasonal discount factors are set to 0.99. To keep the climate-effect comparison simple, the direct-regression coefficients in `MREG` and the instantaneous transfer coefficients ψ_t in `MTF` use discount factor 1, so these coefficient states are static over time. The transfer accumulated-effect state ζ_t retains discount factor 0.99.

The transfer rate λ is selected using the training period only. Guided by an initial coarse scan, the replication script fits the transfer model over $\lambda \in \{0.70, 0.75, 0.80, 0.85, 0.90, 0.95, 0.99\}$ and chooses the value with the smallest posterior predictive loss criterion (PPLC) from

`diagnostics()`. The held-out months are not used in this selection step. The following code shows the selection pattern used by the replication script. The option `exdqlm.max_iter` sets the LDVB iteration cap used in the replication run, so the displayed code records the same convergence control as the executable script.

```
R> lambda.grid = c(0.70, 0.75, 0.80, 0.85, 0.90, 0.95, 0.99)
R> old.opt = options(exdqlm.max_iter = 600L)
R> pplc.grid = rep(NA_real_, length(lambda.grid))
R> for (i in seq_along(lambda.grid)) {
+   set.seed(20264001 + i)
+   temp.MTF = exdqlmTransferLDVB(y = y.train, p0 = 0.15, model = model, df = c(0.99, 0.99),
+     dim.df = c(1, 6), X = X.train, tf.df = c(0.99, 1), lam = lambda.grid[i],
+     tf.m0 = rep(0, 3), tf.C0 = diag(c(0.1, 1, 1), 3), sig.init = 0.1,
+     gam.init = -0.1, n.samp = 1000, tol = 0.05, verbose = FALSE)
+   temp.diag = diagnostics(temp.MTF)
+   pplc.grid[i] = temp.diag$m1.pplc
+ }
R> lambda.star = lambda.grid[which.min(pplc.grid)]
R> lambda.star

[1] 0.99
```

The final models are fit on January 1987 through June 2021. We use the LDVB implementation for all three models; the corresponding MCMC implementation is available when posterior simulation is desired.

```
R> set.seed(20264101)
R> M0 = exdqlmLDVB(y = y.train, p0 = 0.15, model = model, df = c(0.99, 0.99),
+   dim.df = c(1, 6), sig.init = 0.1, gam.init = -0.1,
+   n.samp = 1000, tol = 0.05, verbose = FALSE)
R> reg.comp = regMod(X.train, m0 = rep(0, 2), C0 = diag(1, 2))
R> set.seed(20264301)
R> MREG = exdqlmLDVB(y = y.train, p0 = 0.15, model = model + reg.comp, df = c(0.99, 0.99, 1),
+   dim.df = c(1, 6, 2), sig.init = 0.1, gam.init = -0.1,
+   n.samp = 1000, tol = 0.05, verbose = FALSE)
R> set.seed(20264201)
R> MTF = exdqlmTransferLDVB(y = y.train, p0 = 0.15, model = model, df = c(0.99, 0.99),
+   dim.df = c(1, 6), X = X.train, tf.df = c(0.99, 1), lam = lambda.star,
+   tf.m0 = rep(0, 3), tf.C0 = diag(c(0.1, 1, 1), 3), sig.init = 0.1,
+   gam.init = -0.1, n.samp = 1000, tol = 0.05, verbose = FALSE)
R> options(old.opt)
```

The upper panel of Figure 6 compares the fitted 0.15 quantiles from the three training fits over 2016–2020. We first plot the data, then call `plot()` with `add = TRUE` on the three fitted objects to draw the model fits on common axes.

```
R> par(mfrow = c(3, 1), mar = c(2.8, 4.4, 1.0, 0.9), oma = c(1.8, 0, 0, 0))
R> plot(y.train, col = "grey", ylim = c(1, 8), xlim = c(2016, 2020),
+   ylab = "log flow / quantile")
R> grid(col = "grey90")
R> plot(M0, add = TRUE)
R> plot(MREG, add = TRUE, col = "steelblue")
R> plot(MTF, add = TRUE, col = "forest green")
R> legend("topleft", legend = c("M0 no covariates", "MREG direct regression",
+   "MTF transfer function"), col = c("purple", "steelblue", "forest green"),
+   lty = 1, lwd = 1.5, bty = "n")
```

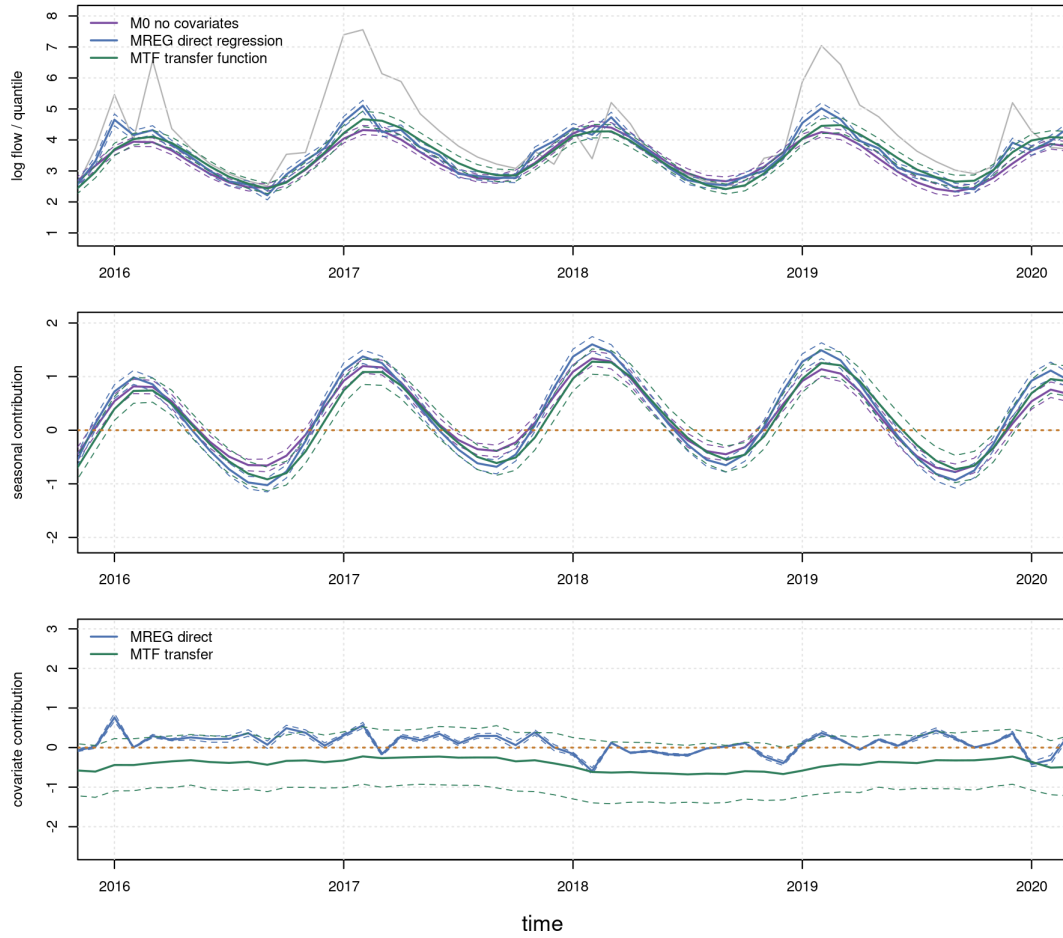


Figure 6: Example 3: Big Tree water flow. Top: posterior mean estimates and 95% CrIs of the fitted 0.15 quantile from the no-covariate baseline M0 (purple), direct-regression model MREG (blue), and transfer-function model MTF (green), plotted with the data (grey) from 2016 to 2020. Middle: posterior mean estimates and 95% CrIs of the combined seasonal components implied by the annual, semiannual, and low-frequency harmonics. Bottom: posterior mean estimates and 95% CrIs of the direct-regression contribution in MREG and the transfer-function contribution in MTF. The horizontal orange dotted lines are at zero for reference.

The fitted lower-tail quantile paths are similar because the trend and seasonal components explain much of the broad annual structure. The component panels of Figure 6 clarify the model differences: the middle panel shows that the seasonal contribution is similar across models, while the lower panel separates the contemporaneous climate-index contribution in MREG from the accumulated transfer contribution in MTF. Component and state views are available from the fitted-object `plot()` method by specifying the parameter `type = "component"` or `type = "state"`, respectively. We begin with the combined seasonal contribution. Because the seasonal block contains three harmonics, this contribution is formed from the second through seventh elements of the state vector. The middle panel of Figure 6 shows the resulting seasonal estimates for the three models over the same 2016–2020 window.

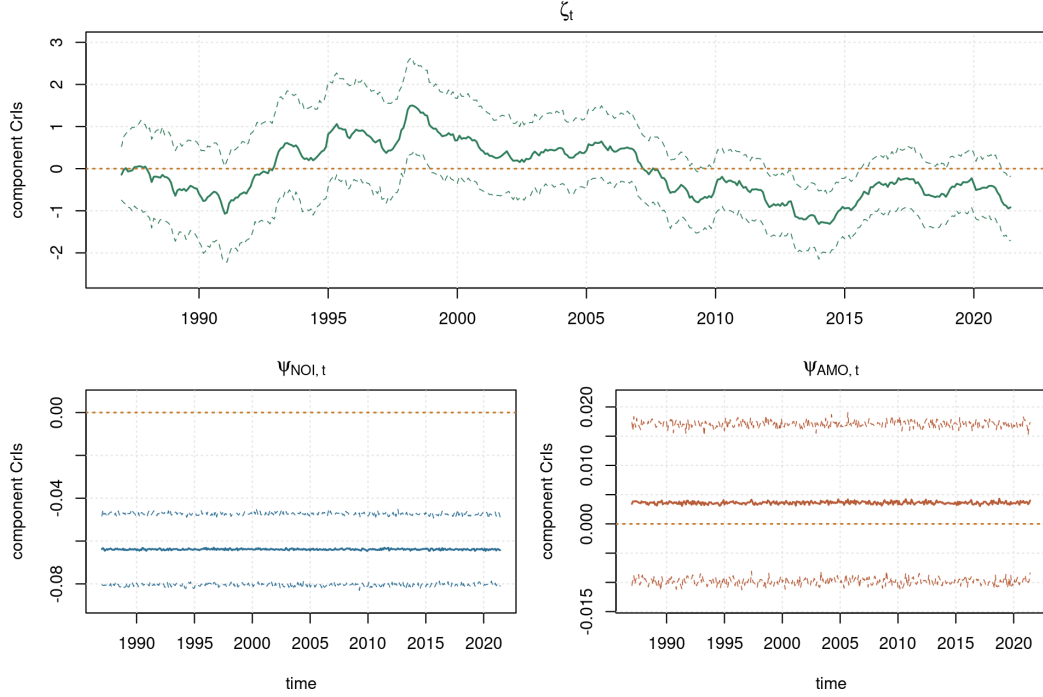


Figure 7: Example 3: Big Tree water flow. Posterior mean estimates and 95% CrIs of the transfer-function states in MTF. The top panel shows the accumulated transfer effect ζ_t . The bottom panels show the instantaneous transfer coefficients $\psi_{\text{NOI},t}$ and $\psi_{\text{AMO},t}$, which are static here because their discount factors are set to 1. Horizontal dotted orange lines mark zero.

```
R> plot(NA, ylim = c(-2, 2), xlim = c(2016, 2020), ylab = "seasonal components")
R> grid(col = "grey90")
R> plot(MO, type = "component", index = 2:7, add = TRUE)
R> plot(MREG, type = "component", index = 2:7, add = TRUE, col = "steelblue")
R> plot(MTF, type = "component", index = 2:7, add = TRUE, col = "forest green")
R> abline(h = 0, col = "orange", lty = 3, lwd = 1.4)
```

The lower panel of Figure 6 displays the climate-index contributions induced by the standardized NOI and AMO covariates. For MREG, indices 8 and 9 are the direct-regression coefficient states. For MTF, index 8 is the accumulated transfer effect ζ_t . This separates contemporaneous and lagged climate-index components from the common trend and seasonal baseline, even when the fitted quantiles themselves are visually close.

```
R> plot(NA, ylim = c(-1.5, 1.5), xlim = c(2016, 2020), ylab = "covariate contribution")
R> grid(col = "grey90")
R> plot(MREG, type = "component", index = 8:9, add = TRUE, col = "steelblue")
R> plot(MTF, type = "component", index = 8, add = TRUE, col = "forest green")
R> abline(h = 0, col = "orange", lty = 3, lwd = 1.4)
R> legend("topleft", legend = c("MREG direct", "MTF transfer"),
+       col = c("steelblue", "forest green"), lty = 1, lwd = 1.5, bty = "n")
```

Figure 7 then displays the augmented transfer states directly. As described in Section 2.1.5, ζ_t controls the accumulated transfer effect and the ψ_t states determine how the contemporaneous

covariates enter that transfer term. In this example the ψ_t discount factors are set to 1, so the bottom panels summarize static NOI and AMO coefficients within the transfer-function model. Setting `type = "state"` in `plot()` plots a single element of the augmented state vector $\tilde{\theta}_t$. In this fitted model, indices 8, 9, and 10 correspond to ζ_t , $\psi_{\text{NOI},t}$, and $\psi_{\text{AMO},t}$, respectively. The NOI coefficient is estimated below zero, while the AMO coefficient is close to zero and its pointwise 95% CrI overlaps zero. We therefore use this analysis to illustrate the specification, inspection, and forecasting of lagged transfer effects, rather than to claim that both climate indices have separately resolved transfer coefficients in these data.

```
R> layout(matrix(c(1, 1, 2, 3), nrow = 2, byrow = TRUE))
R> plot(MTF, type = "state", index = 8, col = "forest green", add = FALSE)
R> grid(col = "grey90")
R> abline(h = 0, col = "orange", lty = 3, lwd = 1.4)
R> title(expression(zeta[t]))
R> par(mar = c(3.8, 4.2, 2.1, 0.8))
R> plot(y.train, type = "n", ylim = c(-0.11, 0.01), xlab = "time", ylab = "component CrIs")
R> plot(MTF, type = "state", index = 9, col = "steelblue", add = TRUE)
R> abline(h = 0, col = "orange", lty = 3, lwd = 1.4)
R> title(expression(psi[list(NOI, t)]))
R> plot(y.train, type = "n", ylim = c(-0.015, 0.020), xlab = "time", ylab = "component CrIs")
R> plot(MTF, type = "state", index = 10, col = "darkorange", add = TRUE)
R> abline(h = 0, col = "orange", lty = 3, lwd = 2)
R> title(expression(psi[list(AMO, t)]))
```

The fitted object also reports `median.kt`, the median number of time steps until the transfer effect on the 0.15 quantile is less than or equal to $1e-3$, as discussed in Section 2.1.5. Here `median.kt` is approximately 350.20, reflecting the highly persistent transfer effect implied by the selected λ at the upper end of the candidate grid.

```
R> MTF$median.kt
```

```
[1] 350.2047
```

We next produce 18-month-ahead forecasts over the held-out window. The future state-space matrices are passed through the `fFF` and `fGG` arguments of the `predict()` method for fitted dynamic objects. This method returns `exdqlmForecast` objects. The code below shows the relevant matrix construction and forecast calls; the replication script wraps these steps in dimension checks and output-writing code. For `M0`, the future state-space matrices repeat the trend and seasonal model.

```
R> k.fore = length(y.holdout)
R> F0.future = model$FF
R> G0.future = model$GG
R> fc.M0 = predict(M0, start.t = length(y.train), k = k.fore, fFF = F0.future,
+               fGG = G0.future, return.draws = TRUE, n.samp = 1000, seed = 20265101)
```

For `MREG`, the future observation matrix includes the held-out NOI and AMO values as direct regressors.

```
R> n.x = ncol(X.holdout)
R> reg.future = regMod(X.holdout, m0 = rep(0, n.x), C0 = diag(1, n.x))
```

```
R> model.reg.future = model + reg.future
R> FREG.future = model.reg.future$FF
R> GREG.future = model.reg.future$GG
R> fc.MREG = predict(MREG, start.t = length(y.train), k = k.fore, fFF = FREG.future,
+                   fGG = GREG.future, return.draws = TRUE, n.samp = 1000, seed = 20265301)
```

For MTF, the future matrices include the held-out NOI and AMO values through the transfer-function augmentation.

```
R> n.state = length(model$m0)
R> FTF.future = matrix(0, nrow = n.state + n.x + 1, ncol = k.fore)
R> FTF.future[seq_len(n.state), ] = F0.future
R> FTF.future[n.state + 1, ] = 1
R> GTF.future = array(0, c(n.state + n.x + 1, n.state + n.x + 1, k.fore))
R> GTF.future[seq_len(n.state), seq_len(n.state), ] = G0.future
R> zeta.ind = n.state + 1
R> psi.ind = n.state + 1 + seq_len(n.x)
R> GTF.future[zeta.ind, zeta.ind, ] = lambda.star
R> for (j in seq_len(n.x)) {
+   GTF.future[zeta.ind, psi.ind[j], ] = X.holdout[, j]
+   GTF.future[psi.ind[j], psi.ind[j], ] = 1
+ }
R> fc.MTF = predict(MTF, start.t = length(y.train), k = k.fore, fFF = FTF.future,
+                   fGG = GTF.future, return.draws = TRUE, n.samp = 1000, seed = 20265201)
```

Finally, Figure 8 overlays the filtered estimates before the forecast origin and the conditional forecasts after it.

```
R> plot(y.fit, col = "grey70", xlim = c(2020, 2023), ylim = c(1,8),
+      ylab = "log flow / forecast quantile", xlab = "time")
R> grid(col = "grey90")
R> plot(fc.M0, add = TRUE, cols = c("purple", "plum"))
R> plot(fc.MREG, add = TRUE, cols = c("steelblue", "lightblue"))
R> plot(fc.MTF, add = TRUE, cols = c("forest green", "darkseagreen"))
R> t.all = as.numeric(time(y.fit))
R> hold.idx = 415:432
R> lines(t.all[hold.idx], y.fit[hold.idx], col = "darkorange", lwd = 1.4)
R> points(t.all[hold.idx], y.fit[hold.idx], col = "darkorange", pch = 1, cex = 0.8)
R> abline(v = t.all[hold.idx[1]], col = "orange", lty = 5, lwd = 1.2)
R> legend("topleft", legend = c("M0 no covariates", "MREG direct regression",
+ "MTF transfer", "held-out observations"),
+       col = c("purple", "steelblue", "forest green", "darkorange"),
+       lty = 1, pch = c(NA, NA, NA, 1), bty = "n")
```

The numerical comparison in Table 8 uses `diagnostics()` so that all reported quantities are exported package diagnostics, as defined in Section 2.2. These diagnostics are computed from the final-training fitted objects.

```
R> diag.M0 = diagnostics(M0)
R> diag.MREG = diagnostics(MREG)
R> diag.MTF = diagnostics(MTF)
R> tab.ex3 = data.frame(model = c("M0", "MREG", "MTF"),
+   KL = c(diag.M0$m1.KL, diag.MREG$m1.KL, diag.MTF$m1.KL),
+   CRPS = c(diag.M0$m1.CRPS, diag.MREG$m1.CRPS, diag.MTF$m1.CRPS),
+   PPLC = c(diag.M0$m1.pplc, diag.MREG$m1.pplc, diag.MTF$m1.pplc))
```

Model	KL	CRPS	PPLC
M0 no covariates	0.122	0.416	177.944
MREG direct regression	0.108	0.347	160.624
MTF transfer function	0.149	0.314	141.694

Table 8: Example 3: Big Tree water flow. Package diagnostics from `diagnostics()` for the no-covariate baseline M0, direct-regression model MREG, and transfer-function model MTF, computed on the final-training fitted objects. KL denotes the deterministic Kullback–Leibler normality diagnostic for the MAP standardized forecast errors, CRPS denotes the continuous ranked probability score, and PPLC denotes the posterior predictive loss criterion. Lower values are preferred; bold entries mark the smallest value for each diagnostic.

Alternatively, Table 9 reports held-out forecast scores over the final 18 months. These diagnostics are computed from the forecast objects using the corresponding `diagnostics()` method. The held-out observations are a required input for this method.

```
R> fc.diag.M0 = diagnostics(fc.M0, y = y.holdout)
R> fc.diag.MREG = diagnostics(fc.MREG, y = y.holdout)
R> fc.diag.MTF = diagnostics(fc.MTF, y = y.holdout)
R> tab.ex3.fc = data.frame(model = c("M0", "MREG", "MTF"),
+   check.loss = c(fc.diag.M0$m1.check_loss, fc.diag.MREG$m1.check_loss,
+   fc.diag.MTF$m1.check_loss),
+   CRPS = c(fc.diag.M0$m1.CRPS, fc.diag.MREG$m1.CRPS, fc.diag.MTF$m1.CRPS))
```

Table 8 gives a mixed in-sample diagnostic comparison. The direct-regression model has the smallest KL value, while the transfer-function model has the smallest CRPS and PPLC values after selecting λ by training-period PPLC. The example shows how **exdqlm** supports three related dynamic quantile workflows: no external inputs, direct regression through `regMod()`, and lagged external-input effects through transfer functions.

Model	Check loss	CRPS
M0 no covariates	0.120	0.473
MREG direct regression	0.114	0.399
MTF transfer function	0.221	0.803

Table 9: Example 3: Big Tree water flow. Held-out forecast metrics over July 2021 through December 2022, computed with `diagnostics()` from `exdqlmForecast` objects returned by `predict(..., return.draws = TRUE)`. Check loss is computed at $p_0 = 0.15$, and CRPS is computed from posterior predictive forecast draws using the integrated-quantile-score approximation. Lower values are preferred; bold entries mark the smallest value in each column.

In the held-out comparison, MREG has the smallest check loss and CRPS, followed by the no-covariate model M0; MTF has the largest value for both metrics. Thus, selecting the transfer parameter λ using training-period PPLC does not make the transfer-function specification the best held-out forecasting model in this example. The purpose of MTF here is to demonstrate how persistent lagged covariate effects can be specified, inspected, and forecast using the package, rather than to claim predictive superiority over direct regression for these data.

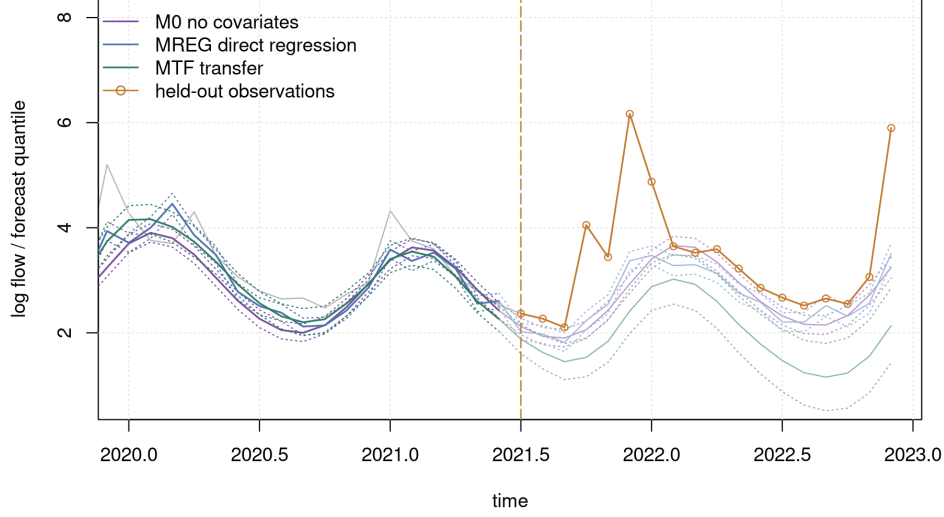


Figure 8: Example 3: Big Tree water flow. Held-out 18-month conditional forecast display from July 2021 through December 2022. The vertical dot-dashed line marks the forecast origin after the final training month, June 2021. Solid lines indicate posterior mean forecast quantiles and dotted lines indicate pointwise 95% CrIs for M0, MREG, and MTF. Held-out observations are shown as orange circles. The forecasts are conditional on the observed holdout-period NOI and AMO values.

4.4. Static exAL regression on a simulated sparse Gaussian benchmark

To illustrate the static routines under sparse shrinkage, we consider a correlated Gaussian regression benchmark for which the target conditional quantile is known exactly. Let

$$\mathbf{x}_i \sim \mathcal{N}_8(\mathbf{0}, \Sigma_x), \quad (\Sigma_x)_{jk} = 0.5^{|j-k|},$$

with sparse signal

$$\boldsymbol{\beta}^* = (3, 1.5, 0, 0, 2, 0, 0, 0)^\top.$$

We use the following benchmark data-generating mechanism. For each target quantile level p_0 , we generate

$$Y_i^{(p_0)} = \mathbf{x}_i^\top \boldsymbol{\beta}^* + \sigma_\varepsilon \{Z_i - \Phi^{-1}(p_0)\}, \quad Z_i \sim \mathcal{N}(0, 1), \quad \sigma_\varepsilon = 1.5.$$

Then

$$Q_Y(p_0 | \mathbf{x}_i) = \mathbf{x}_i^\top \boldsymbol{\beta}^*,$$

so the true p_0 -quantile is the same sparse linear signal at each fitted quantile level. Note that this is a quantile-specific calibration benchmark: each target level is generated so that the known target quantile is the same sparse linear signal, rather than a single-response multi-quantile analysis. The Gaussian data-generating distribution does not coincide with the exAL working likelihood; the benchmark therefore evaluates recovery of a correctly specified conditional quantile under likelihood misspecification. Gaussian design matrices are generated

with `MASS::mvrnorm()` from **MASS** (Venables and Ripley 2002). We use $n = 160$ training observations and an independent holdout sample of size 800.

```
R> Sigma.x = 0.5 ^ as.matrix(dist(1:8))
R> beta.true = c(3, 1.5, 0, 0, 2, 0, 0, 0)
R> rhs.ctrl = list(tau0 = 0.15, zeta2_fixed = 9,
+                 shrink_intercept = FALSE)
R> set.seed(20260712)
R> sim.y = function(X.raw, p0) {
+   drop(X.raw %*% beta.true) + 1.5 * (rnorm(nrow(X.raw)) - qnorm(p0))}
R> X.raw = MASS::mvrnorm(160, mu = rep(0, 8), Sigma = Sigma.x)
R> X = cbind(1, X.raw)
R> X.hold.raw = MASS::mvrnorm(800, mu = rep(0, 8), Sigma = Sigma.x)
R> X.hold = cbind(1, X.hold.raw)
R> ref.hold = drop(X.hold %*% c(0, beta.true))
R> p.grid = c(0.05, 0.25, 0.50)
```

We fit separate static exAL models at $p_0 \in \{0.05, 0.25, 0.50\}$. All fits use the Nishimura–Suchard regularized horseshoe prior (Nishimura and Suchard 2023), implemented in the package through `beta_prior = "rhs_ns"`, with $\tau_0 = 0.15$, fixed slab $\zeta^2 = 9$, and an unshrunk intercept. We fit an LDVB approximation with `exalStaticLDVB()` using `n.samp = 3000`, and an MCMC fit with `exalStaticMCMC()` using `n.burn = 2000` and `n.mcmc = 3000`; the MCMC routine uses its default `collapsed_slice` scale–skewness kernel and is warm-started from the LDVB fit. The lower-tail LDVB fit at $p_0 = 0.05$ converges more slowly, so we allow a larger LDVB iteration budget there. Setting `al.ind = TRUE` (or equivalently `dqlm.ind = TRUE`) would recover the AL special case discussed in Section 2, but here we focus on the general static exAL model under sparse `rhs_ns` shrinkage.

```
R> y.train = y.hold = M.ldvb = M.mcmc = vector("list", length(p.grid))
R> for (i in seq_along(p.grid)) {
+   p0 = p.grid[i]
+   y.train[[i]] = sim.y(X.raw, p0)
+   y.hold[[i]] = sim.y(X.hold.raw, p0)
+   M.ldvb[[i]] = exalStaticLDVB(y = y.train[[i]], X = X, p0 = p0,
+                               beta_prior = "rhs_ns", beta_prior_controls = rhs.ctrl,
+                               max_iter = ifelse(p0 == 0.05, 420, 260),
+                               n.samp = 3000, tol = 1e-4, verbose = FALSE)
+   M.mcmc[[i]] = exalStaticMCMC(y = y.train[[i]], X = X, p0 = p0,
+                               beta_prior = "rhs_ns", beta_prior_controls = rhs.ctrl,
+                               n.burn = 2000, n.mcmc = 3000, thin = 1,
+                               init.from.vb = TRUE, verbose = FALSE)}
```

For static fits, the `diagnostics()` method summarizes fitted quantiles on a common design matrix and, when holdout responses or a reference quantile are supplied, reports predictive evaluation metrics such as check loss and reference-quantile error. It also stores posterior coefficient summaries that can be plotted with `plot(..., type = "coefficients")`. In this simulated example, the true coefficient vector is known, so Figure 9 overlays it as a visual reference. This truth overlay is optional through the parameter `beta.ref` and is not needed for real data applications.

```
R> diag.static = Map(function(ldvb, mcmc, y.h) {
+   diagnostics(ldvb, mcmc, X = X.hold, y = y.h, ref = ref.hold)
```

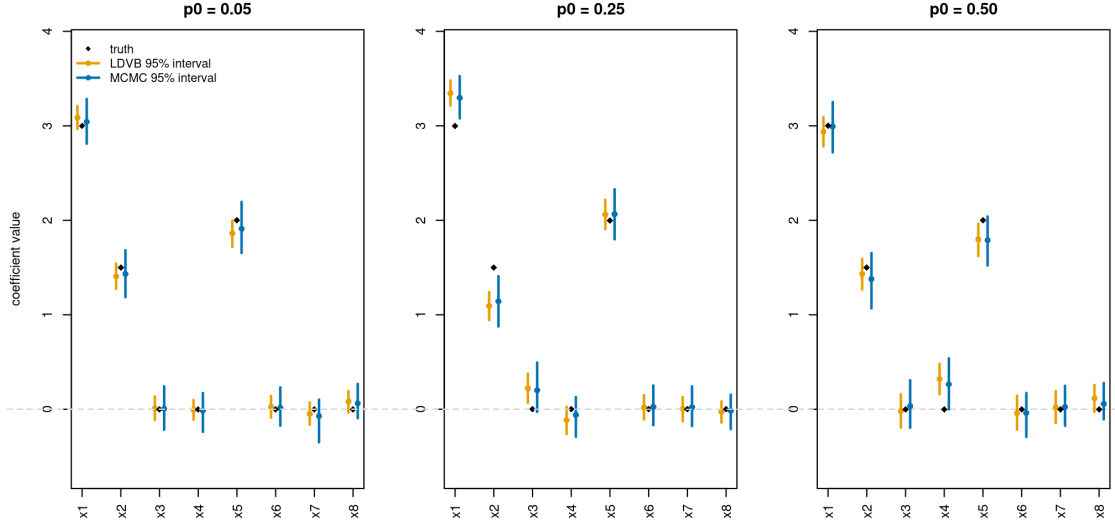


Figure 9: Example 4: sparse static exAL regression under the correlated Gaussian benchmark with the `rhs_ns` prior. Black diamonds denote the true non-intercept coefficients, orange intervals denote LDVB 95% posterior intervals, and blue intervals denote MCMC 95% posterior intervals. Panels correspond to $p_0 = 0.05, 0.25$, and 0.50 .

```

+   }, M.ldvb, M.mcmc, y.hold)
R> y.lim = range(beta.true,
+               unlist(lapply(diag.static, function(z){
+                 c(z$m1.beta.lb[-1], z$m1.beta.ub[-1], z$m2.beta.lb[-1],
+                 z$m2.beta.ub[-1])))))
R> y.lim = y.lim + c(-1, 1) * 0.08 * diff(y.lim)
R> par(mfrow = c(1, 3), mar = c(5.2, 4.0, 2.6, 1.0), xpd = NA)
R> for (i in seq_along(p.grid)) {plot(diag.static[[i]], type = "coefficients",
+   beta.ref = c(0, beta.true), include.intercept = FALSE,
+   coef.names = c("(Intercept)", paste0("x", seq_along(beta.true))),
+   cols = c("orange", "steelblue"), legend.labels = c("LDVB 95% interval",
+   "MCMC 95% interval"), beta.ref.label = "truth", ylim = y.lim,
+   ylab = if (i == 1) "coefficient value" else "",
+   main = sprintf("p0 = %.2f", p.grid[i]), legend = i == 1)}

```

Figure 9 shows that, in this simulated realization, both methods recover the main sparse signal structure and shrink most null coefficients toward zero across the lower tail, lower-middle quantile, and median. Recovery is not uniform across coefficients: the smaller active coefficient is more difficult in the median fit, and a few null-coefficient intervals can miss zero. Across the three panels, the MCMC intervals give cleaner active-signal recovery in the two lower-tail fits, while the median fit is closer; the LDVB intervals provide a faster approximation to the same coefficient-level summaries.

Table 10 includes active-signal RMSE and null-coefficient MAE summaries stored in the `exalStaticDiagnostic` objects returned by `diagnostics()`. The results reinforce the visual comparison in Figure 9. The MCMC posterior-simulation fit has the smaller holdout quantile RMSE in all three fits, although the median difference is small. MCMC gives smaller active-signal RMSE at $p_0 = 0.05$ and $p_0 = 0.25$, while LDVB is slightly smaller at $p_0 = 0.50$.

p_0	method	runtime (s)	active RMSE	null MAE	holdout qRMSE
0.05	LDVB	33.82	0.108	0.034	0.673
	MCMC	82.70	0.069	0.035	0.594
0.25	LDVB	30.19	0.310	0.078	0.371
	MCMC	82.45	0.271	0.065	0.329
0.50	LDVB	13.54	0.128	0.103	0.341
	MCMC	82.90	0.141	0.083	0.335

Table 10: Example 4: runtime and sparse-signal recovery metrics for the static exAL fits under the `rhs_ns` prior. The LDVB rows use `n.samp = 3000`, and the MCMC rows use 2000 burn-in iterations and 3000 retained draws. The active RMSE is computed over the three nonzero coefficients, the null MAE is computed over the five zero coefficients, and the holdout quantile RMSE is computed against the true target quantile on an independent holdout sample of size 800. Because the columns are the comparison metrics, bold entries indicate the smaller value within each quantile block; displayed ties are left unbolded.

Null-coefficient MAE is slightly smaller for LDVB at $p_0 = 0.05$ and for MCMC at $p_0 = 0.25$ and $p_0 = 0.50$. The LDVB fit is nevertheless faster in all three cases, with runtime reductions of roughly 2.4-fold at $p_0 = 0.05$, 2.7-fold at $p_0 = 0.25$, and 6.1-fold at $p_0 = 0.50$. In this benchmark, the LDVB approximation is useful as a rapid first-pass screen, while static MCMC provides a more expensive posterior-simulation comparison when interval accuracy or posterior tail behavior is central.

5. Conclusion

`exdqlm` provides an R workflow for Bayesian quantile regression with primary emphasis on latent state-space models fitted one response quantile at a time. It addresses a gap between broad quantile-regression tools, which do not provide the dedicated latent quantile-state workflow targeted here, and Gaussian dynamic-modeling packages, which do not target AL or exAL quantile likelihoods. Within this setting, `exdqlm` fits DQLMs and exDQLMs, offers MCMC posterior simulation and LDVB approximate inference, and uses the resulting fitted objects for state and component summaries, forecasting, calibration and predictive diagnostics including deterministic KL, CRPS, and PPLC, and posterior-predictive synthesis across separately fitted quantile levels. The package also includes transfer-function state augmentation and static AL/exAL regression with shrinkage priors. These capabilities are exposed through composable model constructors and fitted objects supporting standard methods such as `summary()`, `plot()`, `predict()`, and `diagnostics()`.

The examples illustrate how these interfaces are used in analysis. Lake Huron uses separate dynamic fits at three quantile levels, multi-step forecasting, and post hoc synthesis of posterior predictive draws. Sunspots compares DQLM and exDQLM fits at an upper quantile, uses MAP plug-in one-step-ahead calibration diagnostics together with CRPS and PPLC, and illustrates an in-sample comparison of candidate discount-factor specifications using CRPS, with KL used as a complementary calibration check. Big Tree water flow compares no-covariate, direct-regression, and transfer-function specifications built from a common trend-

seasonal baseline; its fitted and held-out summaries are mixed, so the example is mainly a workflow comparison rather than evidence for a uniformly preferred covariate structure. The static example uses one quantile-specific sparse simulation realization to illustrate LDVB and MCMC under a known target quantile.

The examples also delineate the current scope of the package. Dynamic models are fitted one quantile level at a time. `quantileSynthesis()` combines predictive draws after separate fits and applies monotonicity adjustments when needed; it is not a joint posterior model over multiple quantile levels. LDVB returns samples from fitted variational factors rather than from the exact posterior target, so MCMC remains the reference calculation when posterior tail summaries, interval widths, or approximation error are central. In transfer-function models, lag propagation is controlled by a fixed or grid-selected λ ; the current implementation does not estimate input-specific lag-decay rates internally. Runtime results are elapsed fitting times under the stated benchmark settings and should be read alongside predictive and calibration diagnostics.

Several development directions follow from these limitations: multivariate-output dynamic quantile models that represent cross-series dependence through a modeled correlation matrix for related time series; further assessment of variational approximation error for the nonconjugate (γ, σ) skewness–scale block; nonlinear dynamic quantile modeling beyond the linear state-space form used here; and joint or regularized fitting across quantile levels. The main text, appendices, and replication materials have separate roles: the main text describes the user-facing workflow, the appendices record selected posterior targets and package-specific computational blocks, and the submitted replication script regenerates the reported figures, tables, selected fitted-object output, and benchmark information.

A. Technical overview and notation

The appendix aims to connect the package interface discussed in the article with the posterior targets and computations implemented in **exdqlm**. It records the distributional conventions, compact dynamic and static posterior targets, the LDVB scale-skewness block, the static **rhs_ns** shrinkage block, diagnostic and scoring quantities, and the posterior-predictive synthesis algorithm. Standard Gaussian filtering, smoothing, and backward-sampling recursions are cited rather than rederived. Function-level arguments and backend controls are documented in the package help pages, and the replication materials provide executable code for the reported figures, tables, and benchmark quantities.

A.1. Distributional conventions

Let $p_0 \in (0, 1)$ be the target quantile. The normal distribution $\mathcal{N}(m, V)$ is parameterized by its mean and variance. The inverse-gamma distribution uses the shape-rate convention

$$x \sim \text{IG}(a, b) \iff p(x) = \frac{b^a}{\Gamma(a)} x^{-a-1} \exp(-b/x), \quad x > 0. \quad (\text{A.1})$$

The exponential distribution $\text{Exp}(\text{rate} = 1/\sigma)$ has mean σ . The notation $\mathcal{N}^+(m, V)$ denotes $\mathcal{N}(m, V)$ truncated to $(0, \infty)$.

The asymmetric Laplace (AL) mixture representation used by the package is

$$v \mid \sigma \sim \text{Exp}(\text{rate} = 1/\sigma), \quad (\text{A.2})$$

$$y \mid \eta, \sigma, v \sim \mathcal{N}\{\eta + A(p_0)v, \sigma B(p_0)v\}, \quad (\text{A.3})$$

where η is the target-quantile location and

$$A(p) = \frac{1 - 2p}{p(1 - p)}, \quad B(p) = \frac{2}{p(1 - p)}. \quad (\text{A.4})$$

The extended asymmetric Laplace (exAL) representation adds a latent $s \sim \mathcal{N}^+(0, 1)$. For $\gamma \in (L, U)$, define

$$g(\gamma) = 2\Phi(-|\gamma|) \exp(\gamma^2/2), \quad (\text{A.5})$$

$$p_\gamma = \mathbb{I}(\gamma < 0) + \frac{p_0 - \mathbb{I}(\gamma < 0)}{g(\gamma)}, \quad (\text{A.6})$$

$$A_\gamma = A(p_\gamma), \quad B_\gamma = B(p_\gamma), \quad C_\gamma = \{\mathbb{I}(\gamma > 0) - p_\gamma\}^{-1}. \quad (\text{A.7})$$

The bounds $L < 0 < U$ are the roots $g(L) = 1 - p_0$ and $g(U) = p_0$. Conditional on (v, s, σ, γ) ,

$$y \mid \eta, \sigma, \gamma, v, s \sim \mathcal{N}(\eta + C_\gamma \sigma |\gamma| s + A_\gamma v, \sigma B_\gamma v). \quad (\text{A.8})$$

The package uses $\sigma \sim \text{IG}(a_\sigma, b_\sigma)$ and a proper truncated Student- t prior $\pi_\gamma(\gamma)$ for γ on (L, U) :

$$\pi_\gamma(\gamma) \propto s_\gamma^{-1} t_{\nu_\gamma}\{(\gamma - m_\gamma)/s_\gamma\} \mathbb{I}(L < \gamma < U), \quad (\text{A.9})$$

with the normalizing constant determined by the Student- t distribution function over (L, U) .

The generalized inverse Gaussian (GIG) distribution is parameterized as

$$x \sim \text{GIG}(\lambda, \chi, \psi_{\text{GIG}}) \iff p(x) \propto x^{\lambda-1} \exp \left\{ -\frac{1}{2} \left(\frac{\chi}{x} + \psi_{\text{GIG}} x \right) \right\}, \quad x > 0. \quad (\text{A.10})$$

For $r \in \mathbb{R}$,

$$\mathbb{E}(x^r) = \left(\frac{\chi}{\psi_{\text{GIG}}} \right)^{r/2} \frac{K_{\lambda+r}(\sqrt{\chi\psi_{\text{GIG}}})}{K_{\lambda}(\sqrt{\chi\psi_{\text{GIG}}})}, \quad (\text{A.11})$$

where $K_{\nu}(\cdot)$ is the modified Bessel function of the second kind.

B. Posterior targets and model blocks

B.1. Dynamic DQLM under the AL likelihood

For the dynamic quantile linear model (DQLM), the observation and evolution equations are

$$y_t \mid \boldsymbol{\theta}_t, \sigma, v_t \sim \mathcal{N}(\mathbf{F}_t^\top \boldsymbol{\theta}_t + A(p_0)v_t, \sigma B(p_0)v_t), \quad (\text{B.1})$$

$$\boldsymbol{\theta}_t \mid \boldsymbol{\theta}_{t-1} \sim \mathcal{N}(\mathbf{G}_t \boldsymbol{\theta}_{t-1}, \mathbf{W}_t), \quad (\text{B.2})$$

$$v_t \mid \sigma \sim \text{Exp}(\text{rate} = 1/\sigma). \quad (\text{B.3})$$

The posterior target is proportional to

$$p(\boldsymbol{\theta}_{0:T}, \mathbf{v}, \sigma \mid \mathbf{y}) \propto p(\boldsymbol{\theta}_0) p(\sigma) \prod_{t=1}^T p(y_t \mid \boldsymbol{\theta}_t, \sigma, v_t) p(v_t \mid \sigma) p(\boldsymbol{\theta}_t \mid \boldsymbol{\theta}_{t-1}). \quad (\text{B.4})$$

Conditioning on $(\sigma, \mathbf{v})$, the state sequence is a Gaussian state-space model with observation offset $A(p_0)v_t$ and variance $\sigma B(p_0)v_t$. The package uses standard Kalman filtering, Rauch–Tung–Striebel smoothing, and forward-filtering backward-sampling recursions for this Gaussian block ([West and Harrison 1997](#)).

B.2. Dynamic exDQLM under the exAL likelihood

The dynamic exDQLM augments the AL hierarchy with $s_t \sim \mathcal{N}^+(0, 1)$ and the exAL constants in [\(A.7\)](#):

$$y_t \mid \boldsymbol{\theta}_t, \sigma, \gamma, v_t, s_t \sim \mathcal{N}(\mathbf{F}_t^\top \boldsymbol{\theta}_t + C_\gamma \sigma |\gamma| s_t + A_\gamma v_t, \sigma B_\gamma v_t), \quad (\text{B.5})$$

$$\boldsymbol{\theta}_t \mid \boldsymbol{\theta}_{t-1} \sim \mathcal{N}(\mathbf{G}_t \boldsymbol{\theta}_{t-1}, \mathbf{W}_t), \quad (\text{B.6})$$

$$v_t \mid \sigma \sim \text{Exp}(\text{rate} = 1/\sigma), \quad s_t \sim \mathcal{N}^+(0, 1). \quad (\text{B.7})$$

The posterior target is

$$\begin{aligned} p(\boldsymbol{\theta}_{0:T}, \mathbf{v}, \mathbf{s}, \sigma, \gamma \mid \mathbf{y}) &\propto p(\boldsymbol{\theta}_0) p(\sigma) \pi_\gamma(\gamma) \prod_{t=1}^T p(y_t \mid \boldsymbol{\theta}_t, \sigma, \gamma, v_t, s_t) \\ &\quad \times \prod_{t=1}^T p(v_t \mid \sigma) p(s_t) p(\boldsymbol{\theta}_t \mid \boldsymbol{\theta}_{t-1}). \end{aligned} \quad (\text{B.8})$$

Given $(\sigma, \gamma, \mathbf{v}, \mathbf{s})$, the state sequence is again a Gaussian state-space model, now with offset $A_\gamma v_t + C_\gamma \sigma |\gamma| s_t$ and variance $\sigma B_\gamma v_t$.

B.3. Transfer-function state augmentation

Appendix notation follows the transfer-function formulation in Section 2.1.5. Let $\mathbf{x}_t = (x_{1t}, \dots, x_{kt})^\top$ denote the k -dimensional input vector. The package augments the base dynamic state with a scalar accumulated transfer effect ζ_t and a vector of instantaneous transfer coefficients

$$\boldsymbol{\psi}_t = (\psi_{1,t}, \dots, \psi_{k,t})^\top.$$

The augmented state is

$$\boldsymbol{\theta}_t^{\text{aug}} = (\boldsymbol{\theta}_t^{\text{base}\top}, \zeta_t, \boldsymbol{\psi}_t^\top)^\top, \quad (\text{B.9})$$

with observation row

$$\tilde{\mathbf{F}}_t^\top = (\mathbf{F}_t^\top, 1, \mathbf{0}_k^\top).$$

For fixed transfer rate λ , the transfer block of the evolution matrix is

$$\mathbf{G}_t^{tf} = \begin{pmatrix} \lambda & \mathbf{x}_t^\top \\ \mathbf{0}_k & \mathbf{I}_k \end{pmatrix},$$

so that the scalar accumulated effect evolves as $\zeta_t = \lambda \zeta_{t-1} + \mathbf{x}_t^\top \boldsymbol{\psi}_{t-1} + \omega_t$, while $\boldsymbol{\psi}_t$ follows a random walk with evolution variance specified through the transfer-function discount factors. In Example 4.3, $k = 2$, with normalized NOI and AMO as transfer inputs; the appended transfer states are therefore ζ_t , $\psi_{\text{NOI},t}$, and $\psi_{\text{AMO},t}$.

After this state augmentation, the fitting algorithms are the same dynamic MCMC or LDVB routines used for the corresponding exDQLM.

B.4. Static AL and exAL regression

Let $\mathbf{X} \in \mathbb{R}^{n \times p}$ have rows $\mathbf{x}_i^\top$ and regression coefficient vector $\boldsymbol{\beta}$. Under a Gaussian/ridge prior,

$$\boldsymbol{\beta} \sim \mathcal{N}(\mathbf{b}_{\beta,0}, \mathbf{V}_{\beta,0}). \quad (\text{B.10})$$

The static exAL likelihood is

$$y_i \mid \boldsymbol{\beta}, \sigma, \gamma, v_i, s_i \sim \mathcal{N}(\mathbf{x}_i^\top \boldsymbol{\beta} + C_\gamma \sigma |\gamma| s_i + A_\gamma v_i, \sigma B_\gamma v_i), \quad (\text{B.11})$$

$$v_i \mid \sigma \sim \text{Exp}(\text{rate} = 1/\sigma), \quad s_i \sim \mathcal{N}^+(0, 1). \quad (\text{B.12})$$

The posterior target is

$$p(\boldsymbol{\beta}, \sigma, \gamma, \mathbf{v}, \mathbf{s} \mid \mathbf{y}) \propto p(\boldsymbol{\beta}) p(\sigma) \pi_\gamma(\gamma) \prod_{i=1}^n p(y_i \mid \boldsymbol{\beta}, \sigma, \gamma, v_i, s_i) p(v_i \mid \sigma) p(s_i). \quad (\text{B.13})$$

The AL static regression is recovered by fixing $\gamma = 0$ and dropping the s_i block. For fixed latent variables and scale parameters, the coefficient update is the weighted Gaussian regression update; for `beta_prior = "rhs_ns"`, the Gaussian prior precision is replaced by the expected `rhs_ns` precision described in Section D.

C. LDVB scale–skewness implementation for exAL models

The nonconjugate part of the exAL posterior is the joint scale–skewness block (σ, γ) . The LDVB routines in **exdqlm** specialize nonconjugate variational inference (Wang and Blei 2013) to this block. In version 1.1.1, the default implementation uses a structured $q(\gamma)q(\sigma | \gamma)$ factor that preserves the conditional GIG scale kernel. The earlier two-dimensional Laplace–delta factor on transformed coordinates remains available for comparison and backward-compatible workflows. The dynamic variational family is

$$q(\boldsymbol{\theta}_{0:T}, \mathbf{v}, \mathbf{s}, \sigma, \gamma) = q(\boldsymbol{\theta}_{0:T}) \prod_{t=1}^T q(v_t) \prod_{t=1}^T q(s_t) q_{\sigma\gamma}(\sigma, \gamma). \quad (\text{C.1})$$

The static family replaces $q(\boldsymbol{\theta}_{0:T})$ by $q(\boldsymbol{\beta})$:

$$q(\boldsymbol{\beta}, \mathbf{v}, \mathbf{s}, \sigma, \gamma) = q(\boldsymbol{\beta}) \prod_{i=1}^n q(v_i) \prod_{i=1}^n q(s_i) q_{\sigma\gamma}(\sigma, \gamma). \quad (\text{C.2})$$

The conjugate variational factors require moments of functions of (σ, γ) . In the dynamic case, define

$$\kappa_1 = \mathbb{E}\{1/(\sigma B_\gamma)\}, \quad \kappa_2 = \mathbb{E}\{A_\gamma/(\sigma B_\gamma)\}, \quad \kappa_3 = \mathbb{E}\{A_\gamma^2/(\sigma B_\gamma)\}, \quad (\text{C.3})$$

$$\kappa_4 = \mathbb{E}\{C_\gamma|\gamma|/B_\gamma\}, \quad \kappa_5 = \mathbb{E}\{\sigma C_\gamma^2\gamma^2/B_\gamma\}, \quad \kappa_6 = \mathbb{E}\{A_\gamma C_\gamma|\gamma|/B_\gamma\}. \quad (\text{C.4})$$

For the dynamic state factor, let $m_{1/v,t} = \mathbb{E}(v_t^{-1})$ and $\bar{s}_t = \mathbb{E}(s_t)$. The Gaussian pseudo-observation has precision

$$\phi_t = \kappa_1 m_{1/v,t} \quad (\text{C.5})$$

and pseudo-response

$$y_t^{\text{VB}} = \frac{y_t \phi_t - \kappa_2 - \kappa_4 \bar{s}_t m_{1/v,t}}{\phi_t}. \quad (\text{C.6})$$

Thus the state factor is updated by the same Gaussian state-space recursions as in Section B.1.

For static regression, the coefficient factor is Gaussian. With row vectors $\mathbf{x}_i^\top$,

$$\omega_i = \kappa_1 \mathbb{E}(v_i^{-1}), \quad (\text{C.7})$$

$$\eta_i = y_i \kappa_1 \mathbb{E}(v_i^{-1}) - \kappa_4 \mathbb{E}(v_i^{-1}) \mathbb{E}(s_i) - \kappa_2, \quad (\text{C.8})$$

$$\boldsymbol{\Sigma}_\beta = \left(\mathbf{V}_{\beta,0}^{-1} + \mathbf{X}^\top \text{diag}(\omega_1, \dots, \omega_n) \mathbf{X} \right)^{-1}, \quad (\text{C.9})$$

$$\mathbf{m}_\beta = \boldsymbol{\Sigma}_\beta \left(\mathbf{V}_{\beta,0}^{-1} \mathbf{b}_{\beta,0} + \mathbf{X}^\top \boldsymbol{\eta} \right), \quad (\text{C.10})$$

where $\boldsymbol{\eta} = (\eta_1, \dots, \eta_n)^\top$.

The latent-variable factors remain closed form conditional on the scale–skewness moments. If $m_{\eta,t} = \mathbb{E}(\mathbf{F}_t^\top \boldsymbol{\theta}_t)$, $S_{\eta,t} = \text{Var}(\mathbf{F}_t^\top \boldsymbol{\theta}_t)$, and $S_{\Delta,t} = (y_t - m_{\eta,t})^2 + S_{\eta,t}$, then the dynamic updates are

$$q(v_t) = \text{GIG} \left(\frac{1}{2}, \kappa_1 S_{\Delta,t} - 2\kappa_4 (y_t - m_{\eta,t}) \mathbb{E}(s_t) + \kappa_5 \mathbb{E}(s_t^2), \kappa_3 + 2\mathbb{E}(\sigma^{-1}) \right), \quad (\text{C.11})$$

$$q(s_t) = \mathcal{N}^+(\mu_{s,t}^{\text{VB}}, V_{s,t}^{\text{VB}}), \quad (\text{C.12})$$

$$V_{s,t}^{\text{VB}} = \left(1 + \kappa_5 \mathbb{E}(v_t^{-1}) \right)^{-1}, \quad (\text{C.13})$$

$$\mu_{s,t}^{\text{VB}} = V_{s,t}^{\text{VB}} \left\{ \kappa_4 (y_t - m_{\eta,t}) \mathbb{E}(v_t^{-1}) - \kappa_6 \right\}. \quad (\text{C.14})$$

The static updates replace $m_{\eta,t}$ and $S_{\eta,t}$ by $\mathbf{x}_i^\top \mathbf{m}_\beta$ and $\mathbf{x}_i^\top \boldsymbol{\Sigma}_\beta \mathbf{x}_i$.

The default structured factor works on the bounded logit coordinate

$$z_\gamma = \text{logit} \left(\frac{\gamma - L}{U - L} \right). \quad (\text{C.15})$$

For each candidate value of γ , the conditional factor $q(\sigma \mid \gamma)$ has a GIG kernel. The implementation evaluates the collapsed objective for z_γ , which integrates out σ , and then computes the expectations needed in (C.4) using the quadrature weights for $q(\gamma)$ and GIG moments for $q(\sigma \mid \gamma)$. Approximate posterior draws are obtained by sampling γ from this discrete representation and then drawing σ from the corresponding conditional GIG distribution.

The legacy Laplace–delta factor instead uses transformed coordinates

$$\ell_\sigma = \log \sigma, \quad z_\gamma = \text{logit} \left(\frac{\gamma - L}{U - L} \right).$$

If $r = \text{expit}(z_\gamma)$, its transformed objective is

$$\begin{aligned} h_z(\ell_\sigma, z_\gamma) &= \mathbb{E}_{-(\sigma, \gamma)} \{ \log p(\mathbf{y}, \text{latent variables, parameters}) \} \\ &\quad + \ell_\sigma + \log(U - L) + \log r + \log(1 - r). \end{aligned} \quad (\text{C.16})$$

The legacy factor maximizes this objective, uses the local Hessian to form a Gaussian approximation on (ℓ_σ, z_γ) , and evaluates needed expectations by the delta method. The returned LDVB objects include the scale–skewness factor description, derived moments, ELBO trace, and approximate posterior draws when requested.

The dynamic and static exAL ELBOs have the same block structure:

$$\mathcal{L}(q) = \mathcal{L}_{\text{like}} + \mathcal{L}_{\text{Gaussian}} + \mathcal{L}_v + \mathcal{L}_s + \mathcal{L}_{\sigma\gamma} + H(q_{\text{Gaussian}}) + \sum H\{q(v)\} + \sum H\{q(s)\} + H\{q_{\sigma\gamma}(\sigma, \gamma)\}. \quad (\text{C.17})$$

For dynamic models, the Gaussian block is the state trajectory $q(\boldsymbol{\theta}_{0:T})$; for static models, it is $q(\boldsymbol{\beta})$. The entropy terms use the distributional conventions stated in Section A.1. For AL/DQLM fits, $\gamma = 0$, the s block is omitted, and $q(\sigma)$ is inverse-gamma.

D. Static regression with the Nishimura–Suchard regularized horseshoe

The `rhs_ns` option implements the conditionally conjugate regularized horseshoe formulation of Nishimura and Suchard (2023) for static regression. Let $\mathcal{A} \subseteq \{1, \dots, p\}$ be the set of coefficients subject to shrinkage; by default an intercept is not shrunk. For $j \in \mathcal{A}$,

$$\beta_j \mid \tau^2, \lambda_j^2, \zeta^2 \propto \mathcal{N}(\beta_j \mid 0, \tau^2 \lambda_j^2) \mathcal{N}(\beta_j \mid 0, \zeta^2), \quad (\text{D.1})$$

$$\lambda_j^2 \mid \nu_j \sim \text{IG}(1/2, 1/\nu_j), \quad \nu_j \sim \text{IG}(1/2, 1), \quad (\text{D.2})$$

$$\tau^2 \mid \xi \sim \text{IG}(1/2, 1/\xi), \quad \xi \sim \text{IG}(1/2, 1/\tau_0^2), \quad (\text{D.3})$$

$$\zeta^2 \sim \text{IG}(a_\zeta, b_\zeta), \quad (\text{D.4})$$

unless the slab variance ζ^2 is fixed by the user. The active prior precision contribution is

$$\omega_j^{\text{prior}} = \frac{1}{\tau^2 \lambda_j^2} + \frac{1}{\zeta^2}. \quad (\text{D.5})$$

The static Gaussian coefficient update uses this precision, or its variational expectation, on the active diagonal entries.

Conditioning on β , the shrinkage MCMC block has closed-form updates:

$$\lambda_j^2 \mid \cdot \sim \text{IG} \left(1, \frac{1}{\nu_j} + \frac{\beta_j^2}{2\tau^2} \right), \quad (\text{D.6})$$

$$\nu_j \mid \cdot \sim \text{IG} \left(1, 1 + \frac{1}{\lambda_j^2} \right), \quad (\text{D.7})$$

$$\tau^2 \mid \cdot \sim \text{IG} \left(\frac{|\mathcal{A}| + 1}{2}, \frac{1}{\xi} + \frac{1}{2} \sum_{j \in \mathcal{A}} \frac{\beta_j^2}{\lambda_j^2} \right), \quad (\text{D.8})$$

$$\xi \mid \cdot \sim \text{IG} \left(1, \frac{1}{\tau_0^2} + \frac{1}{\tau^2} \right), \quad (\text{D.9})$$

$$\zeta^2 \mid \cdot \sim \text{IG} \left(a_\zeta + \frac{|\mathcal{A}|}{2}, b_\zeta + \frac{1}{2} \sum_{j \in \mathcal{A}} \beta_j^2 \right), \quad (\text{D.10})$$

with the final line omitted when ζ^2 is fixed.

The mean-field VB block has inverse-gamma factors

$$q(\lambda_j^2), \quad q(\nu_j), \quad q(\tau^2), \quad q(\xi), \quad q(\zeta^2) \text{ if } \zeta^2 \text{ is random.}$$

Let $\bar{\beta}_j^2 = \mathbb{E}_q(\beta_j^2)$. The rate updates are

$$b_{\lambda_j} = \frac{1}{2} \bar{\beta}_j^2 \mathbb{E}\{(\tau^2)^{-1}\} + \mathbb{E}(\nu_j^{-1}), \quad (\text{D.11})$$

$$b_{\nu_j} = 1 + \mathbb{E}\{(\lambda_j^2)^{-1}\}, \quad (\text{D.12})$$

$$b_\tau = \frac{1}{2} \sum_{j \in \mathcal{A}} \bar{\beta}_j^2 \mathbb{E}\{(\lambda_j^2)^{-1}\} + \mathbb{E}(\xi^{-1}), \quad (\text{D.13})$$

$$b_\xi = \frac{1}{\tau_0^2} + \mathbb{E}\{(\tau^2)^{-1}\}, \quad (\text{D.14})$$

$$a_\zeta^* = a_\zeta + \frac{|\mathcal{A}|}{2}, \quad b_\zeta^* = b_\zeta + \frac{1}{2} \sum_{j \in \mathcal{A}} \bar{\beta}_j^2. \quad (\text{D.15})$$

The shape parameters are $a_{\lambda_j} = a_{\nu_j} = a_\xi = 1$ and $a_\tau = (|\mathcal{A}| + 1)/2$.

The **rhs_ns** ELBO is the sum of coefficient-prior, scale-prior, slab-prior, and inverse-gamma entropy terms:

$$\mathcal{L}_{\text{rhs_ns}} = \mathcal{L}_{\beta, \text{hs}} + \mathcal{L}_{\beta, \text{slab}} + \mathcal{L}_{\lambda|\nu} + \mathcal{L}_\nu + \mathcal{L}_{\tau|\xi} + \mathcal{L}_\xi + \mathcal{L}_\zeta + \mathcal{L}_{\text{ent}} + \mathcal{L}_{\beta_0}. \quad (\text{D.16})$$

All terms are available in closed form because each scale factor is inverse-gamma. This block is added to the static AL or exAL ELBO, whose likelihood and scale-skewness components are handled by the surrounding static fitting routine. The fitted static diagnostic object uses the resulting posterior coefficient draws to produce the coefficient-level summaries shown in Example 4.

E. Forecasting, diagnostics, and posterior-predictive synthesis

E.1. Forecasting

At forecast origin $\tilde{t}$, the dynamic state forecast recursion gives

$$\boldsymbol{\theta}_{\tilde{t}+k}^{(m)} \sim \mathcal{N}\{a_{\tilde{t}}^{(m)}(k), R_{\tilde{t}}^{(m)}(k)\}, \quad k = 1, \dots, K,$$

for each posterior or variational draw m . Posterior predictive draws are then generated from

$$Y_{\tilde{t}+k}^{\text{rep},(m)} \sim \text{exAL}_{p_0}\{\mathbf{F}_{\tilde{t}+k}^\top \boldsymbol{\theta}_{\tilde{t}+k}^{(m)}, \sigma^{(m)}, \gamma^{(m)}\}. \quad (\text{E.1})$$

For the AL/DQLM special case, $\gamma^{(m)} = 0$. The function `exdqlmForecast()` implements this recursion, and the standard `predict()` method for dynamic fitted objects calls the same forecast constructor.

E.2. Diagnostics

The fitted dynamic diagnostic sequence is based on maximum a posteriori (MAP) one-step-ahead standardized forecast errors. If $\hat{f}_t$ and $\hat{q}_t$ are the MAP one-step-ahead location and variance, then

$$\hat{e}_t = \frac{y_t - \hat{f}_t}{\sqrt{\hat{q}_t}}, \quad \hat{u}_t = \Phi(\hat{e}_t). \quad (\text{E.2})$$

The diagnostic plot method uses $\{\hat{e}_t\}$ and $\{\hat{u}_t\}$ for the QQ plot, autocorrelation function plot, and standardized forecast-error plot. The reported Kullback–Leibler (KL) values are deterministic one-dimensional calibration diagnostics comparing the empirical distribution of $\{\hat{e}_t\}$ with $N(0, 1)$; the package uses a deterministic normal quantile grid rather than a random reference sample.

For posterior predictive draws $y_t^{(1)}, \dots, y_t^{(M)}$, let $\hat{q}_t(\tau_k)$ be the empirical posterior predictive quantile at level τ_k . The package approximates the continuous ranked probability score (CRPS) by the integrated quantile-score identity

$$\widehat{\text{CRPS}}_t = 2 \sum_{k=1}^K w_k \rho_{\tau_k}\{y_t^{\text{obs}} - \hat{q}_t(\tau_k)\}, \quad (\text{E.3})$$

where the weights sum to one (Gneiting and Raftery 2007; Laio and Tamea 2007). The posterior predictive loss criterion (PPLC) at the fitted quantile level is

$$\widehat{\text{PPLC}} = \sum_t \frac{1}{M} \sum_{m=1}^M \rho_{p_0}\{y_t^{\text{obs}} - y_t^{\text{rep},(m)}\}. \quad (\text{E.4})$$

The held-out forecast diagnostic function applies the same CRPS calculation to forecast objects returned with posterior predictive draws.

E.3. Posterior-predictive synthesis

The function `quantileSynthesis()` combines posterior predictive draws from separately fitted models at ordered levels $p_1 < \dots < p_K$. The result is a post hoc predictive distribution, not a joint multi-quantile posterior.

Algorithm E.1. Posterior-predictive synthesis.

Input: posterior predictive draw matrices or fitted forecast objects from models at ordered quantile levels $p_1 < \dots < p_K$.

Output: synthesized posterior predictive draws and summary intervals on a common predictive scale.

- (1) Extract posterior predictive draw matrices for each fitted quantile.
- (2) For each time point and draw index, form the quantile-specific predictive curve over $p_1, \dots, p_K$.
- (3) Apply the requested monotonicity correction when fitted predictive quantiles cross.
- (4) Interpolate across quantile levels to obtain draws from one predictive distribution.
- (5) Return synthesized posterior predictive draws and summary intervals.

F. Nonconjugate VB and backend notes

For the legacy Laplace–delta factor, let z denote the transformed nonconjugate parameter vector with mode $\hat{z}$ and negative Hessian $\mathbf{H}$ at the mode. The LDVB block uses

$$q(z) = \mathcal{N}(\hat{z}, \mathbf{H}^{-1}). \quad (\text{F.1})$$

For a smooth scalar function $h(z)$,

$$\mathbb{E}\{h(z)\} \approx h(\hat{z}) + \frac{1}{2} \text{tr}\{\nabla^2 h(\hat{z}) \mathbf{H}^{-1}\}. \quad (\text{F.2})$$

The entropy contribution for the transformed Gaussian block is

$$H\{q(z)\} = \frac{1}{2} \log\{(2\pi e)^d |\mathbf{H}^{-1}|\}. \quad (\text{F.3})$$

The Jacobian terms for transforming from (ℓ_σ, z_γ) back to (σ, γ) are included in the objective in (C.16). For the default structured factor, the package combines the entropy of the discrete/quadrature approximation to $q(\gamma)$ with the conditional GIG entropy for $q(\sigma \mid \gamma)$. Other ELBO terms use standard entropy identities for Gaussian, inverse-gamma, GIG, and positive truncated-normal factors under the parameterizations in Section A.1. Those book-keeping terms are implemented in the package and exercised by the package tests rather than rederived here.

The MCMC routines use bounded one-dimensional slice sampling for selected nonconjugate skewness updates. Under the default scale-collapsed kernel, the slice sampler is applied on the bounded logit scale after analytically integrating out σ , and σ is then redrawn from its exact GIG conditional distribution. Legacy slice and MH kernels remain available through `mh.proposal`; their target kernels are the conditional log posteriors induced by the exAL augmentation.

The package also exposes backend options for C++ Kalman recursions, builders, latent-variable samplers, posterior predictive simulation, and MCMC routing. These include the

`exdqlm.use_cpp_*` option family, such as `exdqlm.use_cpp_kf`, and the `exdqlm.cpp_threads` thread cap. They are documented in the package help pages and README because they are runtime controls rather than additional model assumptions.

References

- Barata R, Prado R, Sansó B (2022). “Fast inference for time-varying quantiles via flexible dynamic models with application to the characterization of atmospheric rivers.” *The Annals of Applied Statistics*, **16**(1), 247–271. doi:10.1214/21-A0AS1497.
- Barlow RE, Bartholomew DJ, Bremner JM, Brunk HD (1972). *Statistical Inference Under Order Restrictions: The Theory and Application of Isotonic Regression*. John Wiley & Sons, New York.
- Benoit D, Al-Hamzawi R, Yu K, Van den Poel D (2023). **bayesQR**: *Bayesian Quantile Regression*. R package version 2.4, URL <https://CRAN.R-project.org/package=bayesQR>.
- Bürkner PC (2026). **brms**: *Special Family Functions for brms Models*. Package documentation, accessed 2026-04-19, URL <https://paulbuerkner.com/brms/reference/brmsfamily.html>.
- Carter CK, Kohn R (1994). “On Gibbs sampling for state space models.” *Biometrika*, **81**(3), 541–553.
- Chernozhukov V, Fernández-Val I, Galichon A (2010). “Quantile and Probability Curves Without Crossing.” *Econometrica*, **78**(3), 1093–1125. doi:10.3982/ECTA7880.
- Fan K, Wu C, Ren J, Li X, Zhou F (2026). **pqrBayes**: *Bayesian Penalized Quantile Regression*. R package version 1.2.1, URL <https://CRAN.R-project.org/package=pqrBayes>.
- Fasiolo M, Griffiths B (2025). **qgam**: *Smooth Additive Quantile Regression Models*. R package version 2.0.0, URL <https://CRAN.R-project.org/package=qgam>.
- Frühwirth-Schnatter S (1994). “Data augmentation and dynamic linear models.” *Journal of time series analysis*, **15**(2), 183–202.
- Frumento P (2025). **qrcm**: *Quantile Regression Coefficients Modeling*. R package version 3.2, URL <https://CRAN.R-project.org/package=qrcm>.
- Galarza CE, Benites L, Bourguignon M, Lachos VH (2024). **lqr**: *Robust Linear Quantile Regression*. R package version 5.2, URL <https://CRAN.R-project.org/package=lqr>.
- Gelfand AE, Ghosh SK (1998). “Model choice: a minimum posterior predictive loss approach.” *Biometrika*, **85**(1), 1–11.
- Gneiting T, Raftery AE (2007). “Strictly Proper Scoring Rules, Prediction, and Estimation.” *Journal of the American Statistical Association*, **102**(477), 359–378. doi:10.1198/016214506000001437.

- Gonçalves K, Migon HS, Bastos LS (2017). “Dynamic quantile linear model: a Bayesian approach.” *arXiv preprint arXiv:1711.00162*.
- He X, Pan X, Tan KM, Zhou WX (2023). **conquer**: *Convolution-Type Smoothed Quantile Regression*. R package version 1.3.3, URL <https://CRAN.R-project.org/package=conquer>.
- Huerta G, Jiang W, Tanner MA (2003). “Time series modeling via hierarchical mixtures.” *Statistica Sinica*, pp. 1097–1118.
- Koenker R (2005). *Quantile regression*. Cambridge University Press, New York.
- Koenker R (2025). **quantreg**: *Quantile Regression*. R package version 6.1, URL <https://CRAN.R-project.org/package=quantreg>.
- Kottas A, Krnjajić M (2009). “Bayesian semiparametric modelling in quantile regression.” *Scandinavian Journal of Statistics*, **36**(2), 297–319.
- Kozumi H, Kobayashi G (2011). “Gibbs sampling methods for Bayesian quantile regression.” *Journal of statistical computation and simulation*, **81**(11), 1565–1578.
- Kullback S, Leibler RA (1951). “On information and sufficiency.” *The annals of mathematical statistics*, **22**(1), 79–86.
- Laio F, Tamea S (2007). “Verification Tools for Probabilistic Forecasts of Continuous Hydrological Variables.” *Hydrology and Earth System Sciences*, **11**(4), 1267–1277. doi:10.5194/hess-11-1267-2007.
- Muggeo VMR (2025). **quantregGrowth**: *Non-Crossing Additive Regression Quantiles and Non-Parametric Growth Charts*. R package version 1.7-2, URL <https://CRAN.R-project.org/package=quantregGrowth>.
- Neal RM (2003). “Slice sampling.” *The Annals of Statistics*, **31**(3), 705–767. doi:10.1214/aos/1056562461.
- Nishimura A, Suchard MA (2023). “Shrinkage with shrunken shoulders: Gibbs sampling shrinkage model posteriors with guaranteed convergence rates.” *Bayesian Analysis*, **18**(2), 367–390. doi:10.1214/22-BA1308.
- NOAA Physical Sciences Laboratory (2026). “Climate Indices: Monthly Atmospheric and Ocean Time Series.” URL <https://psl.noaa.gov/data/climateindices/>.
- Ou L, Hunter M, Chow SM, Ji L, Chen M, Hung HJ, Lee J, Li Y, Park J (2021). **dynr**: *Dynamic Models with Regime-Switching*. R package version 0.1.16-2, URL <https://CRAN.R-project.org/package=dynr>.
- Petris G, Gilks W (2018). **dlnm**: *Bayesian and Likelihood Analysis of Dynamic Linear Models*. R package version 1.1-5, URL <https://CRAN.R-project.org/package=dlnm>.
- Plummer M, Best N, Cowles K, Vines K (2006). “CODA: Convergence Diagnosis and Output Analysis for MCMC.” *R News*, **6**(1), 7–11. URL <https://journal.r-project.org/archive/>.

- Prado R, Ferreira MA, West M (2021). *Time Series: Modeling, Computation and Inference*. 2nd edition. Chapman and Hall/CRC Press.
- Prado R, Molina F, Huerta G (2006). “Multivariate time series modeling and classification via hierarchical VAR mixtures.” *Computational Statistics & Data Analysis*, **51**(3), 1445–1462.
- R Core Team (2026). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. URL <https://www.R-project.org/>.
- Reich BJ, Bondell HD, Wang HJ (2009). “Flexible Bayesian quantile regression for independent and clustered data.” *Biostatistics*, **11**(2), 337–352.
- Rosenblatt M (1952). “Remarks on a multivariate transformation.” *The annals of mathematical statistics*, **23**(3), 470–472.
- Sherwood B, Li S, Maidman A (2026). **rqPen**: *Penalized Quantile Regression*. R package version 4.2, URL <https://CRAN.R-project.org/package=rqPen>.
- Taddy MA, Kottas A (2010). “A Bayesian nonparametric approach to inference for quantile regression.” *Journal of Business & Economic Statistics*, **28**(3), 357–369.
- Tokdar S, Cunningham E (2025). **qrjoint**: *Joint Estimation in Linear Quantile Regression*. R package version 2.0-11, URL <https://CRAN.R-project.org/package=qrjoint>.
- US Geological Survey (2016). “National Water Information System data available on the World Wide Web (USGS Water Data for the Nation).” URL <http://waterdata.usgs.gov/nwis/>.
- Venables WN, Ripley BD (2002). *Modern Applied Statistics with S*. Fourth edition. Springer, New York. ISBN 0-387-95457-0, URL <https://www.stats.ox.ac.uk/pub/MASS4/>.
- Wang C, Blei DM (2013). “Variational Inference in Nonconjugate Models.” *Journal of Machine Learning Research*, **14**, 1005–1031. URL <https://jmlr.org/papers/v14/wang13b.html>.
- West M, Harrison J (1997). *Bayesian Forecasting and Dynamic Models*. 2 edition. Springer, New York. doi:10.1007/b98971.
- Yan Y, Zheng X, Kottas A (2025). “A New Family of Error Distributions for Bayesian Quantile Regression.” *Bayesian Analysis*. doi:10.1214/25-BA1507.
- Yu K, Moyeed RA (2001). “Bayesian quantile regression.” *Statistics & Probability Letters*, **54**(4), 437–447.

Affiliation:

Antonio De Leon and Raquel Barata

Department of Statistics

University of California Santa Cruz

Santa Cruz, CA 95064

E-mail: jaguir26@ucsc.edu, raquel.a.barata@gmail.com